



TILERA[®]

***MULTICORE
DEVELOPMENT
ENVIRONMENT
MPIPE
PROGRAMMER'S GUIDE***

DOCUMENT RELEASE 1.40
DOC. NO. UG506
NOVEMBER 2013
TILERA CORPORATION

Copyright © 2011-2013 Tiler® Corporation. All rights reserved. Printed in the United States of America.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, except as may be expressly permitted by the applicable copyright statutes or in writing by the Publisher.

The following are registered trademarks of Tiler Corporation: Tiler and the Tiler logo.

The following are trademarks of Tiler Corporation: Embedding Multicore, The Multicore Company, Tile Processor, TILE Architecture, TILE64, TILEPro, TILEPro36, TILEPro64, TILEExpress, TILEExpress-64, TILEExpressPro-64, TILEExpress-20G, TILEExpressPro-20G, TILEExpressPro-22G, iMesh, TileDirect, TILExtreme-Gx, TILExtreme-Gx Duo, TILEmpower, TILEmpower-Gx, TILEmpower-Gx72, TILEncore, TILEncorePro, TILEncore-Gx, TILEncore-Gx9, TILEncore-Gx16, TILEncore-Gx36, TILEncore-Gx72, TILE-Gx, TILE-Gx9, TILE-Gx16, TILE-Gx36, TILE-Gx72, TILE-Gx3000, TILE-Gx5000, TILE-Gx8000, TILE-Gx8009, TILE-Gx8016, TILE-Gx8036, TILE-Gx3036, DDC (Dynamic Distributed Cache), Multicore Development Environment, Gentle Slope Programming, iLib, TMC (Tiler Multicore Components), hardwall, Zero Overhead Linux (ZOL), MiCA (Multicore iMesh Coprocessing Accelerator), and mPIPE (multicore Programmable Intelligent Packet Engine). All other trademarks and/or registered trademarks are the property of their respective owners.

Third-party software: The Tiler IDE makes use of the BeanShell scripting library. Source code for the BeanShell library can be found at the BeanShell website (<http://www.beanshell.org/developer.html>).

This document contains advance information on Tiler products that are in development, sampling or initial production phases. This information and specifications contained herein are subject to change without notice at the discretion of Tiler Corporation.

No license, express or implied by estoppels or otherwise, to any intellectual property is granted by this document. Tiler disclaims any express or implied warranty relating to the sale and/or use of Tiler products, including liability or warranties relating to fitness for a particular purpose, merchantability or infringement of any patent, copyright or other intellectual property right.

Products described in this document are NOT intended for use in medical, life support, or other hazardous uses where malfunction could result in death or bodily injury.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN "AS IS" BASIS. Tiler assumes no liability for damages arising directly or indirectly from any use of the information contained in this document.

Publishing Information:

Document Number	UG506
Document Release Number	1.40
Date	07 November 2013

Contact Information:

Tiler Corporation
Information info@tilera.com
Web Site http://www.tilera.com

CONTENTS

PREFACE	VII
----------------------	------------

CHAPTER 1 UNDERSTANDING MPIPE

1.1 Overview of mPIPE	1
1.2 Components of mPIPE for Ingress Packet Processing	1
1.2.1 mPIPE Resources	3
1.2.2 mPIPE APIs	3
1.2.3 The Classifier and Its Features	4
1.2.4 mPIPE Context Object	4
1.3 Documentation for mPIPE	4
1.4 Terminology	5

CHAPTER 2 PROGRAMMING AN MPIPE APPLICATION

2.1 Requirements for an Application	7
2.2 mPIPE Application Tasks and Routines	8
2.3 Numbers of Available Resources	8
2.4 Data Structures and Typedefs	9
2.5 Linking with the gxio Library	9

CHAPTER 3 EXAMPLE APPLICATIONS

3.1 Packet Blaster Example	11
3.2 Bridging Example	12
3.3 Chaining Example	14
3.4 Single-Instance Forward Example	15
3.5 Multiple mPIPE Instance Forward Example	17
3.6 Forwarding Fork Example	17
3.7 Ingress Example	18
3.8 Loopback Example	19
3.9 Multi-Rules Example	20
3.10 ping Example	21
3.11 Packet Re-ordering Example	21
3.12 Timestamp Example	23

CHAPTER 4 CLASSIFIER

4.1 Understanding the Classifier	25
4.2 Understanding the Packet Descriptor	25

4.3 Understanding the Default Classifier	27
4.3.1 Understanding Default Classifier Rules	28
4.3.2 Locating the Level 3 and Level 4 Start Values	28
4.4 Location of Classifier files	28
4.5 Using mPIPE with the Simulator	29
CHAPTER 5 CLASSIFIER RULES	
5.1 Overview of Classifier Rules	31
5.2 What Do the Rules Specify?	31
5.3 Data Structures and Typedefs for Rules	32
5.4 Routines for Customizing Rules	32
CHAPTER 6 PROCESSING THE PACKETS	
6.1 Overview of Ingress and Egress Wrappers	35
6.2 Data Structures for Packet Processing	35
6.3 Initializing and Working With an Ingress Queue	35
6.4 Initializing and Working With an Egress Queue	36
6.5 Details About Packet Ordering	36
CHAPTER 7 CUSTOMIZING THE CLASSIFIER	
7.1 Why Modify the Classifier?	37
7.2 Sections of the Classifier	37
7.3 How to Create a Custom Classifier	37
7.4 Using tile-mpipe-cc to Compile a Custom Classifier	38
7.5 Using printf() in the Classifier	38
CHAPTER 8 MANAGING NETWORK LINKS	
8.1 Understanding mPIPE Link Management	39
8.2 Regulating mPIPE Link Permissions	39
8.3 Data Structures for Link Management	40
8.4 Naming Links	40
CHAPTER 9 PROGRAMMING MULTIPLE mPIPE INSTANCES	
9.1 GXIO API Changes for TILE-Gx72	41
9.1.1 Configuring mPIPE Instance Mode	41
9.1.2 Initializing Multiple mPIPE Instance Mode	42
9.1.3 Manipulating mPIPE Buffer Descriptors	42
9.1.4 Allocating Buffer Stacks for Multiple mPIPE Instances	43
9.1.5 Registering Buffers for Multiple mPIPE Instances	43
9.2 Programming for Multiple mPIPE Instances	43

CHAPTER 10 FREQUENTLY ASKED QUESTIONS (FAQS)

10.1 Basic FAQs	47
10.2 FAQs About How to Perform Tasks	48

GLOSSARY 49**INDEX 59**

PREFACE

About This Guide

This guide describes the mPIPE™ (multicore Programmable Intelligent Packet Engine) from Tiler® Corporation from a software perspective.

This guide is a new manual.

This guide applies to MDE Release 4.2.

Intended Audience

This guide is intended for use by Tile Processor application developers.

Note: This guide assumes that you are already familiar with the chapter describing the “mPIPE Architecture” in the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404).

Document Organization

This guide is organized as follows:

- [Chapter 1: Understanding mPIPE](#)
Provides an introduction to mPIPE.
- [Chapter 2: Programming an mPIPE Application](#)
Describes what an application needs to do to take advantage of the mPIPE features.
- [Chapter 3: Example Applications](#)
Describes the mPIPE examples in the MDE Examples directory at `$TILER_ROOT/example/mpipe`.
- [Chapter 4: Classifier](#)
Describes the classifier.
- [Chapter 5: Classifier Rules](#)
Describes the rules used by the classifier.
- [Chapter 6: Processing the Packets](#)
Describes the basic steps in processing packets.
- [Chapter 7: Customizing the Classifier](#)
Provides information on how to customize the classifier.
- [Chapter 8: Managing Network Links](#)
Describes mPIPE physical network link management.

- [Chapter 9: Programming Multiple mPIPE Instances](#)

Describes how to program for multiple instances of the mPIPE engine to support the TILE-Gx72 processor.

- [Chapter 10: Frequently Asked Questions \(FAQs\)](#)

Provides FAQs about how mPIPE and how to use it.

This guide also has a [Glossary](#) and an [Index](#).

MDE Documentation

Note: `TILERA_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is required by many Tiler tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

Tilera MDE Document Index

The Tilera MDE Document Index provides links to online copies of the MDE documentation, in both HTML and PDF formats:

```
$TILERA_ROOT/doc/index.html
```

Location of PDF Files in the Kit

The PDF versions of MDE documents can be found at this location:

```
$TILERA_ROOT/doc/pdf/
```

Man Pages and Info Pages

To view a man page for an x86 tool, run:

```
$ tile-man toolname
```

To view a man page for a Tile tool, run this on a TILE system:

```
$ man toolname
```

To view an info page for a Tile tool, run this on a TILE system:

```
$ info toolname
```

IDE Online Help

The IDE has its own [Tilera IDE Online Documentation](#), which you can access when you use the IDE.

To get to the online help:

- Start `tile-eclipse`.

Select the menu command [Help → Help Contents](#).

- When the online help browser is displayed, select the document [Tilera IDE Online Documentation](#) in the left navigation pane, if it is not already selected.

Technical or Customer Support

You can reach Tilera customer support in either of the following ways:

- Visit the Tiler Web site at <http://www.tilera.com>.
- E-mail questions to support@tilera.com.

Notation Conventions

The following table lists notation conventions used in this guide.

Example	Description
{ }	Alternative required items in syntax descriptions appear within curly brackets. If the contents are separated by a vertical bar, you must choose one of the items.
[]	Optional items in syntax descriptions appear within brackets. If the contents are separated by a vertical bar, you must choose one of the items.
. . .	An ellipse in syntax specifies that the preceding element can be repeated an arbitrary number of times.
\$	This is the system prompt for host-side commands.
#	This is the system prompt for all Tile-side commands and for those x86 host-side commands that must be run as <code>root</code> .
<code>command</code>	A command name appears in the Courier New font
<code>computer output</code>	Computer output appears in the Courier New font.
<i>filename</i>	Non-keyword placeholders appear in italics.
GUI item	A graphical user interface (GUI) item such as a button or menu name appears in red text with the Arial font.
<i>new term</i>	An important word or phrase appears in italics.
Tile Processor™	A processor in the TILE-Gx processor™ family.
TILER_ROOT	TILER_ROOT is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is expected by many Tiler tools and examples. We recommend that you use the <code>tile-env</code> script described in <i>Getting Started with the Multicore Development Environment</i> (UG504) to set it correctly.

CHAPTER 1 UNDERSTANDING MPIPE

This chapter provides an introduction to mPIPE™ (multicore Programmable Intelligent Packet Engine), the networking engine for the Tile Processor:

- [Section 1.1 Overview of mPIPE](#)
- [Section 1.2 Components of mPIPE for Ingress Packet Processing](#)
- [Section 1.3 Documentation for mPIPE](#)
- [Section 1.4 Terminology](#)

See the [Glossary](#) for terms you need to be familiar with.

1.1 Overview of mPIPE

mPIPE is the networking engine for the MDE and TILE-Gx™ processors. It provides these services:

- Packet header parsing to identify the flow of each incoming packet.
- Packet distribution to make ingress packet data available for worker processing.
- Packet Buffer management to track and distribute packet buffer resources.
- Load balancing to spread work across multiple tiles.
- Calculating the L4 checksum on ingress and egress traffic. The software classifier supports IP checksum operations, where as the hardware performs the TCP checksum handling.
- Gathering to collect packet data (potentially scattered across multiple buffers) from tiles.
- Egress services to send packet data to the wire.

mPIPE has these features:

- Wire-speed performance: Over 40 Gbps with one mPIPE on TILE-Gx36™
- Programmable operation in C
- Load balancer can use these modes: round-robin, static flow affinity, dynamic flow affinity, sticky flow affinity
- Use the packet buffer engine to manage buffer sizes, queue types and notification rings
- Classifier has direct access to packet header, descriptor, rules table, and special operations via intrinsics
- Zero overhead on tiles

1.2 Components of mPIPE for Ingress Packet Processing

See [Figure 1-1](#) for an illustration of the basic components of ingress packet processing using mPIPE.

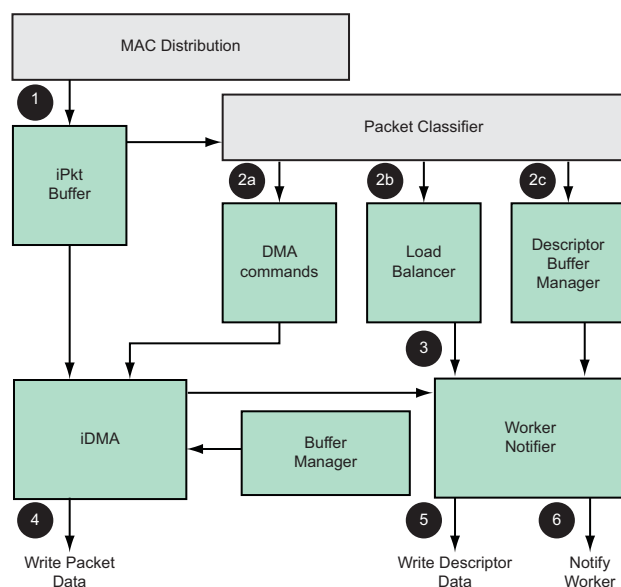


Figure 1-1: Ingress Flow for Packet Processing Using mPIPE

The list below shows what the numbers in the figure mean:

1. A packet is assembled by PHY and MAC layers and presented to channelized iDMA.
2. A packet is classified in order to identify the flow and choose a buffer pool. The classification steps generate a packet descriptor containing the following:
 - a. Buffer pool/DMA control
 - b. Bucket (typically computed from the flow ID)
 - c. Custom fields (for example, flow hash) passed to software
3. The load balancer chooses a worker, based on the hashed flowID created by the classifier.
4. Packet data is written to the chosen buffer pool, typically into the tiles' L3 cache.
5. A packet descriptor is written into a ring in memory space, typically local to the worker.
6. The worker is notified that a new packet descriptor is available.

Table 1-1 shows the major software components relating to the ingress side of mPIPE.

Table 1-1. Software Components Relating to mPIPE

Software Component	Where Located	Where Documented
Classifier	Source: \$TILER_ROOT/ src/sys/mpipe/ classifier.c Binary form: \$TILER_ROOT/ tile/boot/clas- sifier	In this guide and in the source code comments.
mPIPE device API (gxio/mpipe.h)	\$TILER_ROOT/ tile/usr/ include/gxio/ mpipe.h	"gxio Low-Level IO Device Control API" chapter of the <i>Application Libraries Reference Manual</i> (UG527)
mPIPE ingress and egress wrappers	\$TILER_ROOT/ tile/usr/ include/gxio/ mpipe.h	"gxio Low-Level IO Device Control API" chapter of the <i>Application Libraries Reference Manual</i> (UG527)
mPIPE classifier pro- grams (routines for configuring the default classifier pro- gram or loading a custom classifier program)	\$TILER_ROOT/ tile/usr/ include/gxio/ mpipe.h	"gxio Low-Level IO Device Control API" chapter of the <i>Application Libraries Reference Manual</i> (UG527)

1.2.1 mPIPE Resources

The primary mPIPE resources are:

- Notification ring: a region of shared memory, allocated by the application, to which mPIPE delivers packet descriptors. One notification ring is allocated for each worker. Also called NotifRing.
- Notification group: a set of notification rings. Also called NotifGroup.
- Bucket: A container for packet descriptors. The load balancer uses the configuration of the bucket to determine how to distribute packets.
- Buffer stack: a region of shared memory, allocated by the application. The mPIPE allows a maximum of 32 buffer stacks at one time.
- eDMA (egress direct memory access) ring: a region of shared memory, allocated by the application and used to queue packets for egress.

1.2.2 mPIPE APIs

There are several APIs you can use to take advantage of the mPIPE features:

- mPIPE Devices

This API, declared in `gxio/mpipe.h`, is a low-level API that allows applications to configure how Ethernet packets are received and sent. The API is a minimal wrapper around the mPIPE hardware.

The hypervisor uses `gxio_mpipe` to program the classifier in an efficient manner.

- mPIPE ingress and egress wrappers
Convenience routines for receiving packets from a notification ring and sending packets by means of an eDMA ring.
- mPIPE classifier programs
Routines for configuring the default mPIPE classifier program or loading a custom classifier program
- mPIPE link management
Routines for managing physical network links

1.2.3 The Classifier and Its Features

The classifier is a high-speed RISC processor. It has these features:

- C programming interface, which allows users to develop customized programs to take advantage of mPIPE
- Generation of packet descriptors used for load balancing, buffer management, and DMA (direct memory access) control
- L2, L3, and L4 header parsing and validation (with L3 and L4 alignment)
- Linear scaling used in classification (the TILE-Gx36™ processor has 10 classifiers, each running at up to 1.8 GHz)
- Ability to pass custom data to the application
- Direct access to header, descriptor, table, and special operations by means of intrinsics

The software running in the classifier processor is also called the classifier. The classifier code (`classifier.c`) is compiled by `tile-mpipe-cc` and run on the embedded classification processor. See [Chapter 4: Classifier](#).

1.2.4 mPIPE Context Object

All shims with a GXIO interface have a “context” data structure to allow you to use the shim.

For mPIPE, each application initializes an mPIPE context object to help it interact with the mPIPE hardware. The context object is normally shared by all threads. If you program in process mode, you can share the context between a parent process and forked child processes.

1.3 Documentation for mPIPE

See this documentation:

- *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404)
- *Application Libraries Reference Manual* (UG527). These sections:
 - “gxio Low-Level IO Device Control API”:
 - “mPIPE Device API (`gxio/mpipe.h`)”
 - “mPIPE Ingress and Egress Wrappers”
 - “mPIPE Classifier Programs”
 - “mPIPE Link Management”
 - “Examples”:
 - “`mpipe/chaining/app.c`”

- “mpipe/forward/app.c”
- “mpipe/ingress/app.c”
- “mpipe/loopback/app.c”
- “mpipe/multi_rules/app.c”

Also see the examples in `$TILERA_ROOT/examples/mpipe/`.

1.4 Terminology

For basic terminology, see the [Glossary](#).

The hardware and software use different terms to refer to the same thing. See [Table 1-2](#).

Table 1-2. Terminology: Hardware and Software

Hardware Term	Software Term
MPIPE_PDESC_t (ingress packet descriptor)	gxio_mpipe_idesc_t
MPIPE_EDMA_DESC_t (egress packet descriptor)	gxio_mpipe_edesc_t
hpctr/pdptr	getpos/putpos

CHAPTER 2 PROGRAMMING AN MPIPE APPLICATION

This chapter describes what an application needs to do to take advantage of the mPIPE features:

- [Section 2.1 Requirements for an Application](#)
- [Section 2.2 mPIPE Application Tasks and Routines](#)
- [Section 2.3 Numbers of Available Resources](#)
- [Section 2.4 Data Structures and Typedefs](#)
- [Section 2.5 Linking with the gxio Library](#)

2.1 Requirements for an Application

mPIPE needs to keep track of different areas of process (an OS and various applications, for example). It must maintain memory pointers for each notification group (for example, the group's notification ring) or for the range of hash values the group will use. The mPIPE hardware registers to track this information are limited. Thus, each application must allocate the resources it needs to prevent conflicts with other applications.

An application must do some things in order to use mPIPE for ingress and egress packets. In particular:

1. Allocate and initialize the mPIPE resources:
 - Notification rings
 - Notification group
 - Buckets
 - Buffer stack
 - eDMA (egress direct memory access) ring
2. Set up the buffers.
3. Specify at least one rule.
4. Process the packets.

Note: An application does not need to make any changes to the classifier. It can simply use the default classifier.

2.2 mPIPE Application Tasks and Routines

Table 2-1. Tasks Performed and Routines Used in an mPIPE Application

Tasks in the order in which they are generally performed	Routines used
Allocating the notification ring.	<code>gxio_mpipe_alloc_notif_rings()</code>
Initializing the notification ring.	<code>gxio_mpipe_init_notif_ring()</code> or the mPIPE ingress wrapper routine <code>gxio_mpipe_iqueue_init()</code>
Allocating the notification group.	<code>gxio_mpipe_alloc_notif_groups()</code>
Allocating the buckets.	<code>gxio_mpipe_alloc_buckets()</code>
Initializing the notification group and the buckets.	<code>gxio_mpipe_init_notif_group_and_buckets()</code>
Allocating the buffer stack.	<code>gxio_mpipe_alloc_buffer_stacks()</code>
Initializing the buffer stack.	<code>gxio_mpipe_init_buffer_stack()</code>
Allocating the eDMA ring.	<code>gxio_mpipe_alloc_edma_rings()</code>
Initializing the eDMA ring.	<code>gxio_mpipe_init_edma_ring()</code> or the mPIPE egress wrapper routine <code>gxio_mpipe_equeue_init()</code>
Registering memory that contains all the buffers.	<code>gxio_mpipe_register_page()</code>
Pushing buffers onto the stack.	<code>gxio_mpipe_push_buffer()</code>
Specifying the rules.	See Chapter 5: Classifier Rules .
Processing the packets.	See Chapter 6: Processing the Packets .

2.3 Numbers of Available Resources

The list below shows the maximum number of available mPIPE resources:

- Contexts: 16. One context refers to a single service domain. You can have up to 16 independent service domains.
- Buckets: 4160.

The TILE-Gx36™ processor implements 4160 hash buckets. This allows a simple 12-bit hash function to map to the low 4K buckets while reserving 64 buckets for dedicated special purpose flows and applications.

There are two sets of buckets:

- One set from 0 to 4095, allocated in groups of 256
- One set from 4096 to 4160, allocated in groups of four

If an application asks for one, two, three, or four buckets, it gets a group of four.

- Notification rings: 256

- The number of entries for the ingress notification ring and egress ring should be one of four options:
 - 1. Ingress notification ring: 128, 512, 2048, or 65536 descriptors
 - 2. Egress ring: 512, 2048, 8096, or 65536 descriptors.
- Notification groups: 32
- Buffer stacks: 32
- eDMA rings: 24

2.4 Data Structures and Typedefs

These are the data structures and typedefs that applications can use to take advantage of the mPIPE features:

Data structures

- `gxio_mpipe_context_t` (a context object used to manage mPIPE hardware resources)
- `gxio_mpipe_rules_t` (set of classifier rules, plus a context)
- `gxio_mpipe_iqueue_t` (a convenient interface to a notification ring, for use by a single thread)
- `gxio_mpipe_equeue_t` (a convenient, thread-safe interface to an eDMA ring)

Typedefs

- `gxio_mpipe_idesc_t` (ingress packet descriptor; pointer to `MPIPE_PDESC_t`)
- `gxio_mpipe_edesc_t` (egress packet descriptor; pointer to `MPIPE_EDMA_DESC_t`)

2.5 Linking with the gxio Library

To link an application with the `gxio` library, use the `-lgxio` compiler flag.

CHAPTER 3 EXAMPLE APPLICATIONS

This chapter describes the mPIPE examples in the Examples directory at `$TILERA_ROOT/example/mpipe`:

- [Section 3.1 Packet Blaster Example](#)
- [Section 3.2 Bridging Example](#)
- [Section 3.3 Chaining Example](#)
- [Section 3.4 Single-Instance Forward Example](#)
- [Section 3.5 Multiple mPIPE Instance Forward Example](#)
- [Section 3.6 Forwarding Fork Example](#)
- [Section 3.7 Ingress Example](#)
- [Section 3.8 Loopback Example](#)
- [Section 3.9 Multi-Rules Example](#)
- [Section 3.10 ping Example](#)
- [Section 3.11 Packet Re-ordering Example](#)
- [Section 3.12 Timestamp Example](#)

3.1 Packet Blaster Example

This example shows how to use mPIPE to egress packets loaded from a pcap file. It initializes mPIPE, creates an egress queue and a buffer stack, loads the requested pcap file into memory, and registers the pages used for those packets with the buffer stack. Then it sends the packets (at line rate) on the requested link.

This example is located here:

- `$TILERA_ROOT/examples/mpipe/blaster/`

The `app.c` application performs the following tasks involving these functions:

1. Determine the available CPUs through the `tmc_cpus_get_my_affinity()` routine.
2. Use the `tmc_cpus_count()` routine to ensure that you have sufficient worker tiles.
3. Get the instance using the `gxio_mpipe_link_instance()` routine.
4. Use the `gxio_mpipe_init()` routine to start the mPIPE driver.
5. Open the link using the `gxio_mpipe_link_open()` routine.
6. Set the channel using the `gxio_mpipe_link_channel()` routine.
7. Allocate some ingress queues using the `system calloc()` routine.
8. Allocate the number of NotifRings that you need using the `gxio_mpipe_alloc_notif_rings()` routine.

9. Use the `tmc_cpus_find_nth_cpu()` and `tmc_alloc_set_home()` routines to set the CPUs upon we home memory pages.
10. Set the size of the NotifRing using the `tmc_alloc_set_pagesize()` and `tmc_alloc_map()` routines.
11. Initialize the ingress queue using the `gxio_mpipe_iqueue_init()` routine.
12. Allocate one huge page to hold the Egress DMA ring, the buffer stack, all the buffers by calling the `tmc_alloc_get_huge_pagesize()`, `tmc_alloc_set_huge()`, and `tmc_alloc_map()` routines.
13. Call the `gxio_mpipe_alloc_notif_groups()` routine to allocate a NotifGroup.
14. Allocate memory buckets using the `gxio_mpipe_alloc_buckets()` routine.
15. Call the `gxio_mpipe_init_notif_group_and_buckets()` routine in `GXIO_MPIPE_BUCKET_ROUND_ROBIN` mode to initialize the NotifGroup and memory buckets.
16. Allocate an Egress DMA ring through the `gxio_mpipe_alloc_edma_rings()` routine.
17. Initialize the Egress DMA ring through the `gxio_mpipe_equeue_init()` routine.
18. Call the `gxio_mpipe_alloc_buffer_stacks()` routine to create two buffer stacks, one for *large* packets (for egressing the initial packets without having to manually construct chained packets) and one for *small* packets (for ingressing chained packets, and forwarding those packets).
19. Initialize both size stacks through calls to the `gxio_mpipe_init_buffer_stack()` routine after using a `gxio_mpipe_calc_buffer_stack_bytes()` call to aligning the memory to a size of 64K.
20. For both *large* and *small* buffers, register the entire huge page of memory which contains those sized buffers using a `gxio_mpipe_register_page()` call and then populate the stack by calling the `gxio_mpipe_push_buffer()` routine.
21. Register for packets with the mPIPE classifier using the `gxio_mpipe_rules_init()`, `gxio_mpipe_rules_begin()`, `gxio_mpipe_rules_set_headroom()`, and `gxio_mpipe_rules_add_channel()` routines.

3.2 Bridging Example

This example shows how to use mPIPE to perform forwarding on 10 tiles with 40Gbps performance results.

This example is located here:

- `$TILERA_ROOT/examples/mpipe/bridge/`

The `reorder.c` application is based on the `$TILERA_ROOT/examples/mpipe/reorder/app.c` application, and performs the following tasks involving these functions:

1. Determine the available CPUs through the `tmc_cpus_get_my_affinity()` routine.
2. Use the `tmc_cpus_count()` routine to ensure that you have sufficient worker tiles.
3. Get the instance using the `gxio_mpipe_link_instance()` routine.
4. Use the `gxio_mpipe_init()` routine to start the mPIPE driver.
5. Open the link using the `gxio_mpipe_link_open()` routine.

6. Set the channel using the `gxio_mpipe_link_channel()` routine.
7. Allocate some ingress queues using the `system calloc()` routine.
8. Allocate the number of NotifRings that you need using the `gxio_mpipe_alloc_notif_rings()` routine.
9. Initialize the NotifRings using the `gxio_mpipe_iqueue_init()` routine.

Note: To effectively use re-order mPIPE packets, you should use round-robin or sticky affinity modes for load balancing. Thus the total NotifRing (NR) size available for mPIPE is effectively `num_workers * NR-size` before packets start getting dropped. Using a smaller NotifRing size reduces the cache footprint of the application. If you use more than 10 workers on a 40G flow (using round robin or sticky mode) then use an NR-size of 128. Otherwise, use an NR-size of 512 or even 2048 to absorb bursts of packets and latency hiccups on the worker tiles.

10. Set the CPUs upon we home memory pages, using the `tmc_cpus_find_nth_cpu()` and `tmc_alloc_set_home()` routines.
11. Set the size of the NotifRing using the `tmc_alloc_set_pagesize()` and `tmc_alloc_map()` routines.
12. Initialize the ingress queue through the `gxio_mpipe_iqueue_init()` routine.
13. Allocate one huge page to hold the Egress DMA ring, the buffer stack, all the buffers by calling the `tmc_alloc_get_huge_pagesize()`, `tmc_alloc_set_huge()`, and `tmc_alloc_map()` routines.
14. Allocate a NotifGroup by calling the `gxio_mpipe_alloc_notif_groups()` routine.
15. Allocate a single bucket (to get one sequence number space) using a call to the `gxio_mpipe_alloc_buckets()` routine.
16. Initialize the NotifGroup and memory bucket, setting the affinity mode to either Round Robin or Sticky, `gxio_mpipe_init_notif_group_and_buckets()` routine.
17. Create an Egress DMA ring using the `gxio_mpipe_alloc_edma_rings()` routine.

Note: In this packet re-ordering example, there is exactly one sender per link (e.g. per channel).

18. Initialize that Egress DMA ring by calling the `system calloc()` routine and using the `tmc_alloc_set_home()`, `tmc_cpus_find_nth_cpu()`, `tmc_alloc_set_pagesize()`, and `tmc_alloc_map()` routines.
19. Initialize the egress queues using the `gxio_mpipe_equeue_init()` routine.
20. Allocate huge pages to hold both large and small buffers using the `tmc_alloc_get_huge_pagesize()`, `tmc_alloc_set_huge()`, and `tmc_alloc_map()` routines.

Note: This example application sets a large number of small and large buffers to ensure we never run out.

21. Allocate two buffer stacks using the `gxio_mpipe_alloc_buffer_stacks()` routine.
22. Initialize the buffer stacks using the `gxio_mpipe_calc_buffer_stack_bytes()` and `gxio_mpipe_init_buffer_stack()` routines.

23. Register the huge page which contains the buffers by calling the `gxio_mpipe_register_page()` routine.
24. Push some buffers onto the stack using calls to the `gxio_mpipe_push_buffer()` routine with the appropriate memory buffer sizes.
25. Find out what the current SQN is to initialize all of our pointers using the `gxio_mpipe_get_sqn()` routine. There is an additional call to `gxio_mpipe_get_sqn` to support a custom classifier.
26. Register for packets with the mPIPE classifier using the `gxio_mpipe_rules_init()` and `gxio_mpipe_rules_begin()` routines.

The `reorder.c` application allocates and uses global sequence numbers differently from the `reorder/app.c` method.

3.3 Chaining Example

This example shows how to use chained buffers with mPIPE.

The example requests packets on a loopback channel, using small buffers with chaining. It forwards chained buffers transparently.

This example is located here:

- `$TILERA_ROOT/examples/mpipe/chaining/`

The `app.c` application performs the following tasks involving these functions:

1. Use the `tmc_cpus_get_my_affinity()` routine to determine which cores are available.
2. Use the `gxio_mpipe_link_instance()` routine to translate the name of the pipe into the instance number of the mPIPE shim which is connected to that link.
3. Start the mPIPE driver using the `gxio_mpipe_init()` routine.
4. Open the mPIPE link with the `gxio_mpipe_link_open()` routine.
5. Get the channel number with the `gxio_mpipe_link_channel()` routine.
6. Allocate and initialize a NotifRing using the `gxio_mpipe_alloc_notif_rings()` routine.
7. Establish the NotifRings using the `tmc_alloc_set_home()`, `tmc_alloc_set_pagesize()` and `tmc_alloc_map()` routines.
8. Initialize the iqueue using the `gxio_mpipe_iqueue_init()` routine.
9. Allocate one huge page to hold the eDMA ring, the buffer stack, and all the buffers, using the `tmc_alloc_get_huge_pagesize()`, `tmc_alloc_set_huge()`, and `tmc_alloc_map()` routines.
10. Allocate a NotifGroup via the `gxio_mpipe_alloc_notif_groups()` routine.
11. Allocate buckets for the connection using the `gxio_mpipe_alloc_buckets()` routine.
12. Initialize the NotifGroup and the buckets using the `gxio_mpipe_init_notif_group_and_buckets()` routine.
13. Allocate an eDMA ring, using the `gxio_mpipe_alloc_edma_rings()` routine.
14. Initialize the eDMA ring though the `gxio_mpipe_equeue_init()` routine.

15. Use the `gxio_mpipe_alloc_buffer_stacks()` routine to allocate two buffer stacks, one for *large* packets, for egressing the initial packets (instead of hand-constructing chained packets), and one for *small* packets, for ingressing and forwarding chained packets.
16. Separately, for the *large* and *small* buffers, do the following:
 - a. Find the buffer size using the `gxio_mpipe_calc_buffer_stack_bytes()` routine.
 - b. Initialize the buffer stack using the `gxio_mpipe_init_buffer_stack()` routine.
 - c. Register the entire huge page of memory which contains the buffers, using the `gxio_mpipe_register_page()` routine.
 - d. Populate the buffer stacks, using the `gxio_mpipe_push_buffer()` routine.
17. Initialize a classifier program rules list, by calling the `gxio_mpipe_rules_init()` routine.
18. Begin a new rule on that classifier program rules list, using the `gxio_mpipe_rules_begin()` routine.
19. Set the headroom for the rule using the `gxio_mpipe_rules_set_headroom()` routine.
20. Indicate that packets from this channel can be delivered to the buckets and buffer stacks associated with this rule, using the `gxio_mpipe_rules_add_channel()` routine.
21. Commit these channel rules, using the `gxio_mpipe_rules_commit()` routine.

3.4 Single-Instance Forward Example

This example forwards packets.

The packet number is set to 100,000 in the Makefile (`-n 100000`).

It uses the same number of buffers as the number of entries in the eDMA ring. It does this to avoid overflowing the eDMA ring and to allow the packet sequence number to be used to preserve ordering.

See [Figure 3-1](#).

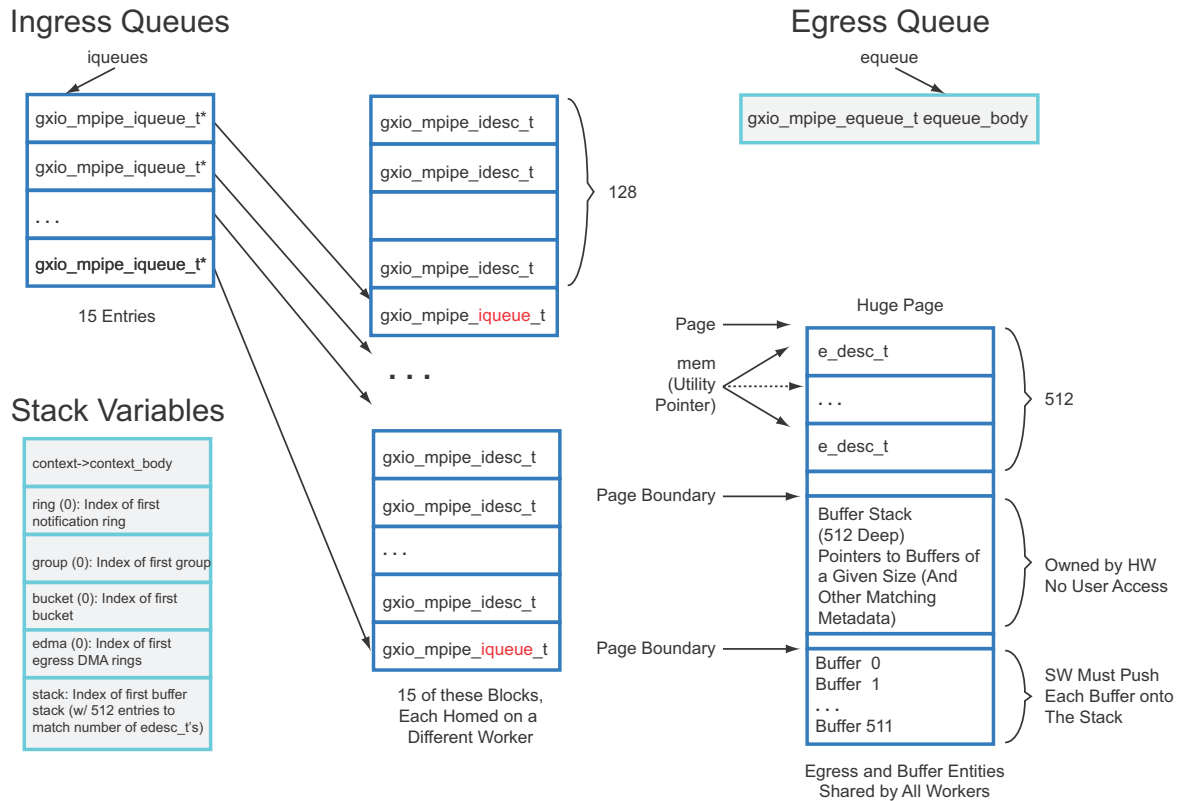


Figure 3-1: Variables and Data Structures from the Forward Example

This example is located here:

- `$TILERA_ROOT/examples/mpipe/forward/`
- "mpipe/forward/app.c" in the "Example Documentation" chapter of the *Application Libraries Reference Manual* (UG527)

The `app.c` application performs the following tasks involving the functions in the `gxio_mpipe` API:

1. Sets up the number of packets to process by default: 1000.
2. Initializes the mPIPE driver and obtains an mPIPE context using `gxio_mpipe_init(gxio_mpipe_context_t *)`. The context is shared by all workers.
3. Establishes one ingress queue (`gxio_mpipe_iqueue_t`) for each worker. Each ingress queue points to a block of packet descriptors (`gxio_mpipe_idesc_t` objects). Each block is homed on a different worker and is used by an mPIPE notification ring:
 - a. Allocates some notification rings (mPIPE resources) using `gxio_mpipe_alloc_notif_rings()`.
 - b. Initializes the notification rings using `gxio_mpipe_iqueue_init()`.
4. Allocates one huge page to hold the eDMA ring, the buffer stack, and all the packets.
5. Configures the packet distribution (round-robin) and balancing:

- a. Allocates a notification group (mPIPE resource) using `gxio_mpipe_alloc_notif_groups()`.
 - b. Allocates a bucket (mPIPE resource) using `gxio_mpipe_alloc_buckets()`.
 - c. Initializes the notification group and the bucket using `gxio_mpipe_init_notif_group_and_buckets()`.
6. Establishes one egress queue (`gxio_mpipe_equeue_t`) shared by all workers:
 - a. Allocates an eDMA ring (mPIPE resource) using `gxio_mpipe_alloc_edma_rings()`.
 - b. Initializes the egress queue using `gxio_mpipe_equeue_init()`.
7. Allocates a buffer stack using `gxio_mpipe_alloc_buffer_stacks()`.
8. Initializes the buffer stack using `gxio_mpipe_init_buffer_stack()`.
9. Registers the huge page of memory using `gxio_mpipe_register_page()`.
10. Pushes some buffers onto the stack using `gxio_mpipe_push_buffer()`.
11. Requests all incoming packets (but drops jumbo packets):
 - a. Initializes the rules using `gxio_mpipe_rules_init()`.
 - b. Commits the rules using `gxio_mpipe_rules_begin()`, and `gxio_mpipe_rules_commit()`.
12. Creates multiple threads to pull in packets and forward them.
13. Uses `sim_enable_mpipe_links()` to allow packets to flow (for simulator use only).
14. Get packet descriptors using `gxio_mpipe_iqueue_try_peek()` and `gxio_mpipe_iqueue_peek()`.
15. Forwards the packets:
 - a. Egresses the packets using `gxio_mpipe_equeue_put_at()`.
 - b. Informs the hardware after each packet has been processed using `gxio_mpipe_iqueue_consume()`.

3.5 Multiple mPIPE Instance Forward Example

This example forwards packets using two instances of mPIPE.

This example is located here:

- `$TILERA_ROOT/examples/mpipe/forward/`

For details on the `app_gx8072.c` example, see [Section 9.2 Programming for Multiple mPIPE Instances](#).

3.6 Forwarding Fork Example

This example shows how to use mPIPE with the `fork()` routine. It initializes mPIPE. It requests all incoming packets (but drops jumbo packets). It forks multiple processes to pull in packets and forward them.

This example is located here:

- `$TILERA_ROOT/examples/mpipe/forward_fork/`

The `app.c` application performs the following tasks involving these functions:

1. Determine the available CPUs through the `tmc_cpus_get_my_affinity()` routine.
2. Get the instance using the `gxio_mpipe_link_instance()` routine.
3. Start the mPIPE driver using the `gxio_mpipe_init()` routine.
4. Allocate some Ingress Queues by calling the `tmc_alloc_map()` routine.
5. Allocate some NotifRings through the `gxio_mpipe_alloc_notif_rings()` routine.
6. Initialize the NotifRings, by calling the `tmc_alloc_set_shared()`, `tmc_alloc_set_home()`, `tmc_cpus_find_nth_cpu()`, `tmc_alloc_set_pagesize()`, `tmc_alloc_map()` and `gxio_mpipe_iqueue_init()` routines.
7. Allocate one hugepage to hold the edma ring, the buffer stack, and all the packets through the `tmc_alloc_get_huge_pagesize()` and `tmc_alloc_set_huge()` routines.
8. Allocate a NotifGroup using the `gxio_mpipe_alloc_notif_groups()` routine.
9. Allocate some memory buckets through the `gxio_mpipe_alloc_buckets()` routine.
10. Initialize the NotifGroup and the memory buckets, preserving packet order among flows (setting the bucket mode to `GXIO_MPIPE_BUCKET_ROUND_ROBIN`) using the `gxio_mpipe_init_notif_group_and_buckets()` routine.
11. Allocate an Egress DMA ring through the `gxio_mpipe_alloc_edma_rings()` routine.
12. Initialize that Egress DMA ring using the `gxio_mpipe_equeue_init()` routine.
13. Allocate a buffer stack through the `gxio_mpipe_alloc_buffer_stacks()` routine.
14. Initialize the buffer stack, using the `gxio_mpipe_calc_buffer_stack_bytes()` and `gxio_mpipe_init_buffer_stack()` routines.
15. Register the entire huge page of memory which contains all the buffers using the `gxio_mpipe_register_page()` routine.
16. Push some buffers onto the stack through the `gxio_mpipe_push_buffer()` routine.
17. Register for packets by initializing the mPIPE rules system using the `gxio_mpipe_rules_begin()`, `gxio_mpipe_rules_commit()`, and `gxio_mpipe_rules_init()` routines.
18. Synchronize the system using the `tmc_sync_barrier_init()` routine.
19. Await packets using the `tmc_task_watch_forked_children()` routine.

3.7 Ingress Example

This example uses the `gxio_mpipe` API to receive Ethernet packets by means of the mPIPE shim.

This example is located here:

- `$TILERA_ROOT/examples/mpipe/ingress/`
- “`mpipe/ingress/app.c`” in the “Example Documentation” chapter of the *Application Libraries Reference Manual* (UG527)

The `app.c` application performs the following tasks involving the functions in the `gxio_mpipe` API:

1. Starts the driver using `gxio_mpipe_init()`. In this routine, `mpipe_index` is the index of the physical mPIPE device. The return value is 0 if successful.
2. Allocates a notification ring (NotifRing) using `gxio_mpipe_alloc_notif_rings()`.

General note about this routine: `count` must be less than the largest contiguous chunk of available rings. `first` is ignored unless flags include `*FIXED`, and must be 0 to the hardware limit (511). `first` plus `count` must be less than the hardware limit.

3. Initializes the notification ring using `gxio_mpipe_iqueue_init()`.
4. Allocates a notification group using `gxio_mpipe_alloc_notif_groups()`.
5. Allocates a bucket using `gxio_mpipe_alloc_buckets()`.
6. Initializes the notification group and the bucket using `gxio_mpipe_init_notif_group_and_buckets()`.
7. Allocates a buffer stack using `gxio_mpipe_alloc_buffer_stacks()`.
8. Initializes the buffer stack using `gxio_mpipe_init_buffer_stack()`.
9. Registers the entire huge page of memory that contains all the buffers using `gxio_mpipe_register_page()`.
10. Pushes some buffers onto the stack using `gxio_mpipe_push_buffer()`.
11. Specifies rules for the packets using `gxio_mpipe_rules_commit()`.
12. Turns on all the links on `mpipe0` using `sim_enable_mpipe_links()` (for simulator use only).
13. Processes the packets using `gxio_mpipe_iqueue_get()`.
14. Pushes the buffer using `gxio_mpipe_push_buffer()`.

3.8 Loopback Example

This example shows how to use chained buffers in an mPIPE application.

This example is located here:

- `$TILERA_ROOT/examples/mpipe/loopback/`

The `app.c` application performs the following tasks involving the functions in the `gxio_mpipe` API:

1. Initializes mPIPE.
2. Requests packets on a loopback channel (but drops jumbo packets).
3. Seeds the loopback channel with some packets.
4. Creates multiple threads to forward incoming packets.
1. Find out which CPUs you might have available using the `tmc_cpus_get_my_affinity()` routine.
2. Start the mPIPE driver using the `gxio_mpipe_init()` routine.
3. Open the mPIPE link with the `gxio_mpipe_link_open()` routine.
4. Allocate a set of NotifRings through the `gxio_mpipe_alloc_notif_rings()` routine.
5. For each NotifRing, initialize a NotifRing, specifying memory, size and flags using a `gxio_mpipe_init_notif_rings()` routine.
6. Use the `tms_alloc_map()` routine to allocate one huge page to hold the eDMA ring, the buffer stack, and all the buffers.
7. Create a NotifGroup with the `gxio_mpipe_alloc_notif_groups()` routine.

8. Allocate buckets for data using the `gxio_mpipe_alloc_buckets()` routine.
9. Initialize both NotifRings and buckets by calling the `gxio_mpipe_init_notif_group_and_buckets()` routine. This causes the load balancer to run the best combination of load balancing functions according to your input.
10. Allocate a set of eDMA rings using the `gxio_mpipe_alloc_edma_rings()` routine.
11. Initialize those eDMA rings by calling the `gxio_mpipe_equeue_init()` routine.
12. Allocate a set of buffer stacks with the `gxio_mpipe_alloc_buffer_stacks()` routine.
13. Initialize those buffers using the `gxio_mpipe_init_buffer_stack()` routine.
14. Register the entire page of memory which contains all the buffers, using the `gxio_mpipe_register_page()` routine.
15. Push a buffer onto the buffer stacks by calling the `gxio_mpipe_push_buffer()` routine.
16. Initialize a classifier program rules list, by calling the `gxio_mpipe_rules_init()` routine.
17. Begin a new rule on that classifier program rules list, using the `gxio_mpipe_rules_begin()` routine.
18. Initialize a barrier to the state in which no tasks have arrived at the barrier, through the `tmc_sync_barrier_init()` routine.

3.9 Multi-Rules Example

This example shows how to set up multiple mPIPE rules. It shows how to use mPIPE rules for the incoming PCAP packets under the simulator. The packet numbers being processed by each tile with a certain mPIPE rule are shown. The rules include rules for port filtering, VLAN filtering, and MAC filtering, combined with different flow control attributes like round-robin, static flow, and dynamic flow.

This example is located here:

- `$TILERA_ROOT/examples/mpipe/multi_rules/`

The `app.c` application performs the following tasks involving these functions:

1. Use the `tmc_cpus_get_my_affinity()` routine to determine which cores are available.
2. Check available cpu number using the `tmc_cpus_count()` routine.
3. Use the `gxio_mpipe_link_instance()` routine to translate the name of each “pipe” into the instance number of the mPIPE shim which is connected to that link.
4. Start the mPIPE driver using the `gxio_mpipe_init()` routine.
5. Open each of the three mPIPE links with the `gxio_mpipe_link_open()` routine.
6. Define each channel uniquely using separate calls to the `gxio_mpipe_link_channel()` routine.
7. Allocate and initialize one NotifRing for each worker using the `gxio_mpipe_alloc_notif_rings()` routine.
8. For each channel, now do the following:
9. Set the `mmap_flags` value to represent the desired cache homing using the `tmc_alloc_set_home()` routine.
 - a. Use the `tmc_alloc_set_pagesize()` routine to set the NotifRing size.

- b. Use the `tmc_alloc_map()` routine to allocate one ingress queue for each worker tile to use processing the channel.
 - c. Initialize the `NotifRing` for each channel using the `gxio_mpipe_iqueue_init()` routine.
4. Allocate memory to the three `NotifGroups` by calling the `gxio_mpipe_alloc_notif_groups()` routine.
5. Allocate buckets using the `gxio_mpipe_alloc_buckets()` routine.
6. Allocate one buffer stack for all of the channels by calling the `gxio_mpipe_alloc_buffer_stacks()` routine.
7. Use the `tmc_alloc_get_huge_pagesize()` routine to determine how much memory you need.
8. Set the `MAP_HUGETLB` bit in the `mmap_flags` word using the `tmc_alloc_set_huge()` routine.
9. Allocate the actual memory that you need by calling the `tmc_alloc_map()` routine.
10. Use the `gxio_mpipe_calc_buffer_stack_bytes()` routine Calculate the number of bytes required to store a given number of buffers in the memory registered with a buffer stack.
11. Initialize the buffers using the `gxio_mpipe_init_buffer_stack()` routine.
12. Register the huge memory page through the `gxio_mpipe_register_page()` routine.
13. Push buffers onto the buffer stacks by calling the `gxio_mpipe_push_buffer()` routine.
14. Initialize a classifier program rules list, by calling the `gxio_mpipe_rules_init()` routine.
15. For each `NotifGroup` and bucket set, do the following:
 - a. Initialize a `NotifGroup` and range of buckets for the load balancer using the `gxio_mpipe_init_notif_group_and_buckets()` routine.
 - b. Begin a new rule on that classifier program rules list, using the `gxio_mpipe_rules_begin()` routine.
16. Commit all classifier rules by calling the `gxio_mpipe_rules_commit()` routine.

3.10ping Example

This example shows how to connect two simulator instances by means of `tile-monitor`.

Using the *peered simulator* commands, each simulator's mPIPE shim is configured to connect to the other simulator's mPIPE shim. This capability is particularly useful for developing applications that require dynamic header generation rather than static PCAP (packet capture) files.

This example is located here:

- `$TILERA_ROOT/examples/mpipe/ping/`

For more information, see the `README.txt` file in the `ping` directory.

3.11Packet Re-ordering Example

This example shows how to use a software queue to reorder packets coming from mPIPE. This allows you to process a single flow in round-robin mode on different worker tiles without losing the original packet order.

It initializes mPIPE. It requests all incoming packets (but drops jumbo packets). It creates multiple threads to pull in packets and forward them. It forwards any number of links using a configurable-size pool of workers. The application forwards packets out the link on which they were received, but you could change this to be a lookup table to perform L2/3 forwarding. You can set the size of the reorder queue based on the application requirements and additional services can be added either in the worker or in the send-function. Any code that you add to the worker should generally provide an upper bound on latency to prevent blocking; use the worker code to identify the egress port and potentially also Quality of Service.

Note: Code that you add on the SEND side of the software queue can have longer/unbounded latency without blocking thus it is better to implement traffic management application code on the SEND side.

This example is located here:

- \$TILERA_ROOT/examples/mpipe/reorder/

The app.c application performs the following tasks involving these functions:

1. Determine the available CPUs through the `tmc_cpus_get_my_affinity()` routine.
2. Use the `tmc_cpus_count()` routine to ensure that you have sufficient worker tiles.
3. Get the instance using the `gxio_mpipe_link_instance()` routine.
4. Use the `gxio_mpipe_init()` routine to start the mPIPE driver.
5. Open the link using the `gxio_mpipe_link_open()` routine.
6. Set the channel using the `gxio_mpipe_link_channel()` routine.
7. Allocate some ingress queues using the `system_calloc()` routine.
8. Allocate the number of NotifRings that you need using the `gxio_mpipe_alloc_notif_rings()` routine.
9. Initialize the NotifRings using the `gxio_mpipe_iqueue_init()` routine.

Note: To effectively use re-order mPIPE packets, you should use round-robin or sticky affinity modes for load balancing. Thus the total NotifRing (NR) size available for mPIPE is effectively `num_workers * NR-size` before packets start getting dropped. Using a smaller NotifRing size reduces the cache footprint of the application. If you use more than 10 workers on a 40G flow (using round robin or sticky mode) then use an NR-size of 128. Otherwise, use an NR-size of 512 or even 2048 to absorb bursts of packets and latency hiccups on the worker tiles.

10. Set the CPUs upon we home memory pages, using the `tmc_cpus_find_nth_cpu()` and `tmc_alloc_set_home()` routines.
11. Set the size of the NotifRing using the `tmc_alloc_set_pagesize()` and `tmc_alloc_map()` routines.
12. Initialize the ingress queue through the `gxio_mpipe_iqueue_init()` routine.
13. Allocate one huge page to hold the Egress DMA ring, the buffer stack, all the buffers by calling the `tmc_alloc_get_huge_pagesize()`, `tmc_alloc_set_huge()`, and `tmc_alloc_map()` routines.
14. Allocate a NotifGroup by calling the `gxio_mpipe_alloc_notif_groups()` routine.

15. Allocate a single bucket (to get one sequence number space) using a call to the `gxio_mpipe_alloc_buckets()` routine.
16. Initialize the NotifGroup and memory bucket, setting the affinity mode to either Round Robin or Sticky, `gxio_mpipe_init_notif_group_and_buckets()` routine.
17. Create an Egress DMA ring using the `gxio_mpipe_alloc_edma_rings()` routine.

Note: In this packet re-ordering example, there is exactly one sender per link (e.g. per channel).

18. Initialize that Egress DMA ring by calling the system `calloc()` routine and using the `tmc_alloc_set_home()`, `tmc_cpus_find_nth_cpu()`, `tmc_alloc_set_pagesize()`, and `tmc_alloc_map()` routines.
19. Initialize the egress queues using the `gxio_mpipe_equeue_init()` routine.
20. Allocate huge pages to hold both large and small buffers using the `tmc_alloc_get_huge_pagesize()`, `tmc_alloc_set_huge()`, and `tmc_alloc_map()` routines.

Note: This example application sets a large number of small and large buffers to ensure we never run out.

21. Allocate two buffer stacks using the `gxio_mpipe_alloc_buffer_stacks()` routine.
22. Initialize the buffer stacks using the `gxio_mpipe_calc_buffer_stack_bytes()` and `gxio_mpipe_init_buffer_stack()` routines.
23. Register the huge page which contains the buffers by calling the `gxio_mpipe_register_page()` routine.
24. Push some buffers onto the stack using calls to the `gxio_mpipe_push_buffer()` routine with the appropriate memory buffer sizes.
25. Find out what the current SQN is to initialize all of our pointers using the `gxio_mpipe_get_sqn()` routine.
26. Register for packets with the mPIPE classifier using the `gxio_mpipe_rules_init()` and `gxio_mpipe_rules_begin()` routines.

3.12 Timestamp Example

The example shows how to access the mPIPE timestamp registers.

This example is located here:

- `$TILERA_ROOT/examples/mpipe/timestamp/`

The file `app.c` provides code to access the mPIPE timestamp registers through the user-space GXIO libraries. You can use this method when you manage the native network interfaces by either the Linux kernel or by the user space application.

For more information, see the `README.txt` file in the `timestamp` directory.

The file `testptp.c` provides code to test the IEEE 1388 Precision Time Protocol (PTP) from user space.

The `app.c` application performs the following tasks involving these functions:

1. Get the mPIPE instance number by calling the `gxio_mpipe_link_instance()` routine on the `link_name`.

2. Initialize the mPIPE driver using the `gxio_mpipe_init()` function with the mPIPE context and instance.
3. Get the current time-stamp using the `gxio_mpipe_get_timestamp()` routine.
4. Set the mPIPE time from the system time using the `gxio_mpipe_set_timestamp()` routine with the mPIPE context.
5. Adjust the mPIPE timestamp clock by feeding the mPIPE context and adjusted time to the `gxio_mpipe_adjust_timestamp()` routine.

CHAPTER 4 CLASSIFIER

This chapter describes the classifier:

- [Section 4.1 Understanding the Classifier](#)
- [Section 4.2 Understanding the Packet Descriptor](#)
- [Section 4.3 Understanding the Default Classifier](#)
- [Section 4.4 Location of Classifier files](#)
- [Section 4.5 Using mPIPE with the Simulator](#)

4.1 Understanding the Classifier

The mPIPE uses a programmable packet classifier. The classifier generates a packet descriptor based on the packet headers of the incoming packet.

This packet descriptor identifies the flow for load balancing and ordering, controls where the data is written, and identifies exception flows. The classifier parses all of the L2 and some or all of the L3 headers in order to identify, for example, the IP source and destination and determine the L4 octet offset.

The mPIPE uses up to 10 parallel classification engines. Since packet classification is stateless, there is no communication required between the engines and the performance scales linearly with the addition of more classification engines.

Hardware in the mPIPE ensures that the packet order is maintained through classification, hence the implementation of many parallel classifiers appears to the user as a single high-speed classification.

The classifier does the following for each packet:

1. Parses the packet headers
2. Encodes some packet metadata in the *ingress packet descriptor* (the `gxio_mpipe_idesc_t`, also known as the *idesc*).
3. Either drops the packet or continues further processing of the packet.
4. If it continues processing the packet, picks a notification ring to handle the packet and a buffer stack to contain the packet.

4.2 Understanding the Packet Descriptor

The packet descriptor is generated by the packet classifier.

[Figure 4-1](#) shows the components of a packet descriptor.

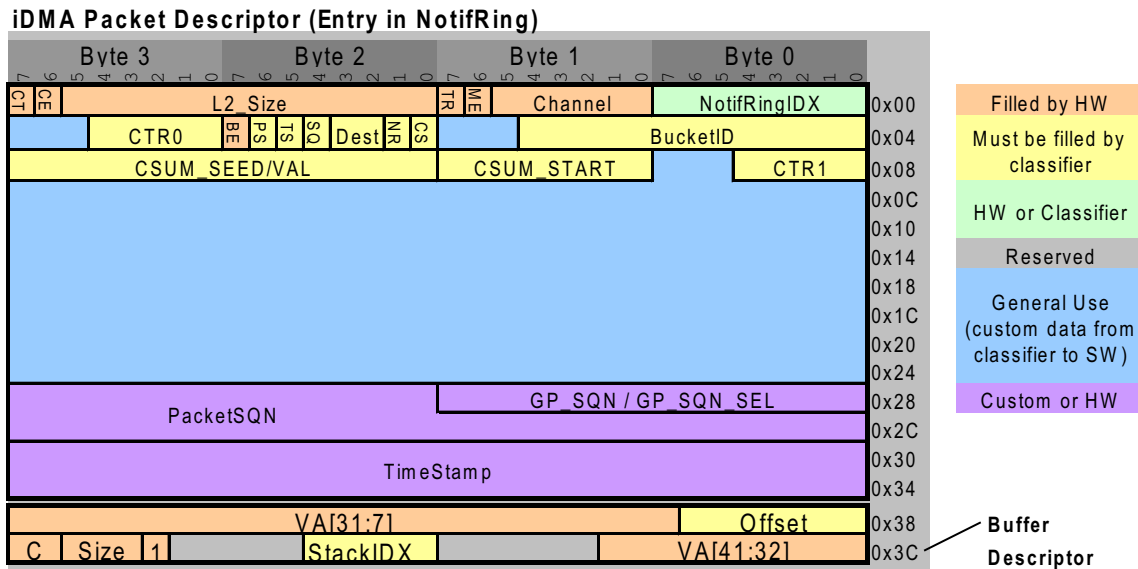


Figure 4-1: Packet Descriptor

The packet descriptor has categories of fields:

- The following fields are set by the hardware:
 - Channel: Source channel.
 - ME: MAC error. This condition generally occurs only if the mPIPE clock is running too slowly.
 - TR: Truncate error. The packet was truncated due to lack of space. There is no space in the mPIPE ingress buffer (192K).
 - L2_Size: Final L2 size of the packet. The L2_size does not include preamble or CRC unless those fields are enabled for pass-thru from the MAC layer.
 - CE: L2 CRC error. Generated by the MAC. Asserted if the MAC indicated an L2 CRC error or other L2 error (bad length, and so forth) on the packet.
 - CT: Cut-through. Cut-through is used in certain configurations when store-and-forward is impractical due to area, power, or bandwidth constraints.
 - C (Chained), Size, VA: Filled by stack manager based on buffer chosen from StackIDX.
- The following fields are set by either the hardware or the classifier:
 - NotifRingIDX. Notification ring to write `gxio_mpipe_idesc_t` into. Typically filled by the load balancer, but can be overridden by classifier with the NR bit set.
- The following fields are set by the default classifier:
 - BucketID: Bucket identifier.
 - CS: If set, checksum generation enabled.
 - NR: If set, NotifRingIDX is going to be determined by the classifier instead of the load balancer.
 - Dest:

- 0: Drop packet, drop PDESC (idesc).
- 1: Write packet to buffer(s) from BufferIDX.
- 2: Write PDESC, drop packet.
- 3: RSVD.
- SQ: Sequence number selector information. When this bit is clear, the GP_SQN and GP_SQN_SEL fields are available for custom use.
- TS: Enable TimeStamp insertion.
- PS: Enable PacketSQN insertion.
- CTR0: Encoded counter selects.
- CTR1: Encoded counter selects.
- CSUM_Start: Start byte for checksum.
- CSUM_SEED and VAL: Initial seed for checksum (from the classifier); later filled with CSUM result by hardware.
- Offset: Start offset within the buffer for the packet data.
- StackIDX: Buffer stack to use for this packet.
- PacketSQN: Packet sequence number.
- You may set the following fields by defining a custom classifier:
 - TimeStamp: Timestamp assigned at arrival from MAC.
 - GP_SQN and GP_SQN_SEL: These fields are set by the classifier and are blank by default. These fields are available for custom use when the SQ bit (set by the classifier) is clear.

Note: For detailed information about the fields in the packet descriptor, see the “mPIPE Architecture” chapter in the *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404).

4.3 Understanding the Default Classifier

The default binary classifier chooses which notification ring and buffers stack to use based upon the following criterion for packets:

- Channel
- VLAN
- Destination MAC address
- Flow hash
- Packet size

The channel, VLAN, and destination MAC address are used to pick a rule. The rule is used to pick a buffer stack that will hold the packet (based on the packet size) and a bucket (based on the flow hash). The bucket is used by the load balancer to determine which application will receive the packet.

By default, a rule matches all channels, VLANs, and destination MACs. The programmer can add restrictions as needed.

By default, the classifier supports all the rules from all the applications.

4.3.1 Understanding Default Classifier Rules

The default binary classifier uses *rules*, one of which is selected based on the channel, VLAN, and dMAC of incoming packets, and then the rule determines the bucket, offset, and buffer stack to be used for the packet.

Each rule specifies *bucket* information, which is combined with the *flow hash* to determine the actual bucket.

Each rule specifies a headroom and a tailroom, and a capacity. If the total size (packet size plus headroom plus tailroom) exceed the capacity, the packet will be dropped. Otherwise, the buffer stack will be chosen from an array of eight buffer stacks provided by the rule, based on which standard buffer size index (128, 256, 512, 1024, 1664, 4096, 10368, or 65535) can contain the total size. The offset will be the headroom (plus any required chaining pointer).

The default classifier program, has a rule that matches all channels, VLANs, and destination MACs.

The capacity can be used to ensure that chaining will not occur even if an unexpectedly large packet arrives.

Next comes two hash tables, implemented as arrays of entries, where each entry is a key, and a mask of the rules which allow that key, except that the high bit of `rules_hi` indicates a *collision*, that is, IF the current entry is NOT a match, the next entry must be consulted. Thus, the high bit of `rules_hi` is not available for representing an actual rule. We avoid having to *wrap* while scanning through collisions by duplicating some initial entries at the end of the array.

For more information about the rules, see [Chapter 5: Classifier Rules](#).

4.3.2 Locating the Level 3 and Level 4 Start Values

From the file `$TILERA_ROOT/tile/usr/tilera/src/sys/mpipe/classifier.c`, the default packet classifier uses values that are defined in the structure `idesc`.

The classifier uses these values:

```
// Actual "idesc" (aka "pDesc") contents (in little endian order):
...
// 0x0C - 0x0F: custom0: flow hash
// 0x10 - 0x11: custom1: vlan (or 0xFFFF)
// 0x12 - 0x13: custom1: ethertype
// 0x14 - 0x14: custom1: l3 start (Starting address for Layer 3, for all
packets)
// 0x15 - 0x15: custom1: l4 start (Starting address for Layer 4, for IPv4/IPv6
packets, else zero)
// 0x16 - 0x16: custom1: protocol (for IPv4/IPv6 packets, else zero)
// 0x17 - 0x17: custom1: status (0x80 = bad IPv4 header checksum, packet status)
// 0x18 - 0x1F: custom2
// 0x20 - 0x27: custom3
```

Use the `gxio_mpipe_iqueue_drop_if_bad()` function to access the `custom1` value for packet status.

4.4 Location of Classifier files

The classifier is shipped as a binary program in this location:

```
$TILERA_ROOT/tile/boot/classifier
```

This file is automatically included in bootroms created by `tile-monitor`, and is automatically loaded by the hypervisor at boot time.

The actual classifier code (which you can modify if you choose) is here:

```
$TILERA_ROOT/src/sys/mpipe/classifier.c
```

The hypervisor mPIPE driver is here:

```
$TILERA_ROOT/src/sys/hv/tilegx/drivers/mpipe/classifier.c
```

To load the classifier, use either of these routines:

- `gxio_mpipe_classifier_load_from_file()`
- `gxio_mpipe_classifier_load_from_bytes()`

4.5 Using mPIPE with the Simulator

The simulator includes mPIPE simulation.

To use the simulator, you need to put this call in your application:

```
// Allow packets to flow.
sim_enable_mpipe_links(0, -1);
```

You can use the simulator option `link_enable` to enable flow. For more information, see the “TILE-Gx Processor Simulator (tile-sim)” in the MDE Document Index.

CHAPTER 5 CLASSIFIER RULES

This chapter describes the rules used by the classifier:

- [Section 5.1 Overview of Classifier Rules](#)
- [Section 5.2 What Do the Rules Specify?](#)
- [Section 5.3 Data Structures and Typedefs for Rules](#)
- [Section 5.4 Routines for Customizing Rules](#)

5.1 Overview of Classifier Rules

A channel is a set of physical resources in the mPIPE. Each MAC is associated with one or more channels. Depending on system configuration, a channel might be shared by more than one MAC.

The channel, VLAN, and destination MAC (dMAC) are used to pick a rule in the classification stage of ingress packet processing.

The rule is used to pick a buffer stack that will hold the packet (based on the packet size), a bucket (based on the flow hash), and an offset into the bucket where the packet will start. The bucket is used by the load balancer to determine which application will receive the packet.

An application can use the default rules that are called in the default classifier program. By default, a rule matches all channels, VLANs, and destination MACs. However, most applications will want to receive traffic matching a particular channel/VLAN/dMAC pattern. Thus, an application will need to change the rules to allocate its own buffer stacks and load balancer buckets, and map traffic to those stacks and buckets.

A set of rules is known as a *rules list*.

An empty rule matches all packets.

An empty rules list matches no packets.

The programmer will need to use the functions regarding rules in the `gxio/mpipe.h` API.

The rules are passed to the hypervisor, which combines them with the rules from other applications.

The maximum number of rules is limited to 31 across all applications (including Linux). The actual limit might be less if any rules are extremely complex.

The maximum size of a rule is variable, but users will not normally exceed the size limit. However, if you made a rule that, for example, matched 3,000 different VLANs, you might exceed the size limit.

5.2 What Do the Rules Specify?

Each rule specifies:

- Headroom

- Tailroom
- Capacity

The buffer stack is chosen from an array of eight buffer stacks provided by the rule. These buffer stack size indexes are:

- 128
- 256
- 512
- 1024
- 1664
- 4096
- 10368
- 65535

The buffer stack is chosen based on which standard buffer size index can contain the total size.

The offset is the headroom plus any required chaining pointer.

If the total size (packet size plus headroom plus tailroom) exceed the capacity, the packet will be dropped.

The capacity can be used to ensure that chaining will not occur even if an unexpectedly large packet arrives.

For example, a rule could specify a headroom of 66 and a tailroom of 32. It could then could request that packets of up to 512 minus (66 plus 32) bytes should use a buffer stack of 512-byte buffers, and that packets of up to 1664 minus (66 plus 32) bytes should use a buffer stack of 1664-byte buffers. It should specify a capacity of 1664 to force larger packets to be dropped. Or it should specify a larger capacity to allow chaining to be done using the 1664-byte buffers.

5.3 Data Structures and Typedefs for Rules

These are the data structures and typedef that applications can use for customizing classifier rules:

Data structures

- `gxio_mpipe_rules_t` (set of classifier rules, plus a context)
- `gxio_mpipe_rules_stacks_t` (eight buffer stack IDs)
- `gxio_mpipe_rules_dmac_t` (a destination MAC address)

Typedef

- `uint16_t` (a VLAN)

5.4 Routines for Customizing Rules

These are the routines you can use to customize rules:

- To load a classifier program binary, use either the `gxio_mpipe_classifier_load_from_file()` or `gxio_mpipe_classifier_load_from_bytes()` routines.
- To initialize a classifier program rules list, use the `gxio_mpipe_rules_init()` routine.

- To begin a new rule on the rules list, use the `gxio_mpipe_rules_begin()` routine.
- To set the priority of a rule, use the `gxio_mpipe_rules_set_priority()` routine.
- To set the headroom of a rule, use the `gxio_mpipe_rules_set_headroom()` routine.
- To set the tailroom of a rule, use the `gxio_mpipe_rules_set_tailroom()` routine.
- To set the capacity of a rule, use the `gxio_mpipe_rules_set_capacity()` routine.
- To specify that packets from a particular channel can be delivered to the buckets and buffer stacks associated with the current rule, call the `gxio_mpipe_rules_add_channel()` routine.
- To specify that packets targeting a particular destination MAC address can be delivered to the buckets and buffer stacks associated with the current rule, use the `gxio_mpipe_rules_add_dmac()` routine.
- To specify that packets with a particular VLAN can be delivered to the buckets and buffer stacks associated with the current rule, call the `gxio_mpipe_rules_add_vlan()` routine.
- To commit the rules, call the `gxio_mpipe_rules_commit()` routine.

CHAPTER 6 PROCESSING THE PACKETS

This chapter describes the rules used by the classifier:

- [Section 6.1 Overview of Ingress and Egress Wrappers](#)
- [Section 6.2 Data Structures for Packet Processing](#)
- [Section 6.3 Initializing and Working With an Ingress Queue](#)
- [Section 6.4 Initializing and Working With an Egress Queue](#)
- [Section 6.5 Details About Packet Ordering](#)

6.1 Overview of Ingress and Egress Wrappers

The `gxio/mpipe.h` API provides convenience routines for receiving packets from a notification ring and sending packets by means of an eDMA ring. These routines are the mPIPE ingress and egress wrappers.

The mPIPE ingress and egress hardware uses shared memory packet descriptors to describe packets that have arrived on ingress or are destined for egress. These descriptors are stored in shared memory ring buffers and are written or read by hardware as necessary. The wrapper routines manage the head and tail pointers for these rings, allowing a programmer to easily read and write packet descriptors.

6.2 Data Structures for Packet Processing

These are the data structures and typedefs that applications can use with the ingress and egress wrappers:

Data structures

- `gxio_mpipe_iqueue_t` (an interface to a notification ring, for use by a single thread)
- `gxio_mpipe_equeue_t` (a thread-safe interface to an eDMA ring)

6.3 Initializing and Working With an Ingress Queue

To initialize an ingress queue, the application passes the following to the `gxio_mpipe_iqueue_init()` routine:

- `gxio_mpipe_iqueue_t` state object
- Ring number from `gxio_mpipe_alloc_notif_ring()`
- Address in memory to hold a ring buffer

Once initialized, the `gxio_mpipe_iqueue_t` API provides two flows for getting the `gxio_mpipe_idesc_t` packet descriptor associated with incoming packets.

The simpler method is to call `gxio_mpipe_iqueue_get()` or `gxio_mpipe_iqueue_try_get()`. These routines copy the oldest packet descriptor out of the notification ring and into a descriptor provided by the caller. They also immediately inform the hardware that a descriptor has been processed.

For applications with stringent performance requirements, you can achieve higher efficiency by avoiding the packet descriptor copy and processing of multiple descriptors at the same time. In this case, you would use the `gxio_mpipe_iqueue_peek()` and `gxio_mpipe_iqueue_try_peek()` routines.

No locking is required for ingress queues.

For more information about these optimization routines, see the “mPIPE Ingress and Egress Wrappers” section of the *Application Libraries Reference Manual* (UG527).

6.4 Initializing and Working With an Egress Queue

Applications can create multiple egress queues, but there is no way to configure their priority.

To initialize an egress queue, the application passes the following to the `gxio_mpipe_equeue_init()` routine:

- a `gxio_mpipe_equeue_t` state object
- the ring number from the `gxio_mpipe_alloc_edma_rings()` routine
- the shared memory buffer which holds the egress descriptors (the number of egress descriptors must be 512, 2048, 8192, or 16384).

Once initialized, the `gxio_mpipe_equeue_t` API provides two flows for getting the `gxio_mpipe_edesc_t` packet descriptor associated with outgoing packets.

The simpler method is to call `gxio_mpipe_equeue_put()`. This routine allows the programmer to wait for an eDMA ring slot to become available and write a single descriptor into the ring.

An alternative method is to use the `gxio_mpipe_equeue_reserve()` and `gxio_mpipe_equeue_put_at()` routines to reserve multiple eDMA ring slots and then fill each slot with a `gxio_mpipe_edesc_t` packet descriptor. This allows you to reduce per-operation overhead by posting multiple packets with a single `gxio_mpipe_equeue_reserve()` call. It also allows gather operations to be performed by posting multiple descriptors, one for each fragment in the final egress packet.

No locking is required for egress queues.

You can have 24 egress queues for the TILE-Gx36™.

For more information about these routines, see the “mPIPE Ingress and Egress Wrappers” section of the *Application Libraries Reference Manual* (UG527).

6.5 Details About Packet Ordering

Every incoming packet gets a global sequence number, which can be used by the application to maintain ordering. If the application is the only one using mPIPE, then this can be done simply.

Dropped packets are ignored, and thus have no impact on packet order.

Non-fragmented packets can be distributed to workers based on flow hash. Thus, individual flows can be easily kept in order.

Currently, the classifier does not flow hash fragmented packets.

CHAPTER 7 CUSTOMIZING THE CLASSIFIER

This chapter provides information on how to customize the classifier:

- [Section 7.1 Why Modify the Classifier?](#)
- [Section 7.2 Sections of the Classifier](#)
- [Section 7.3 How to Create a Custom Classifier](#)
- [Section 7.4 Using tile-mpipe-cc to Compile a Custom Classifier](#)
- [Section 7.5 Using printf\(\) in the Classifier](#)

Note: This chapter uses the term “classifier” to refer to the software classifier.

7.1 Why Modify the Classifier?

You can use the classifier without modification.

In general, modifications will be done in the first third of the classifier, where the initial analysis is done. You could add support for new headers, extract and store new metadata, or add entirely new code paths. An example of a new code path would be for special processing of loopback packets.

7.2 Sections of the Classifier

There are three main sections to the `classifier.c` file:

- The first section analyzes the packet.
- The second section chooses which application wants the packet.
- The third section chooses the bucket and buffer for the packet.

In general, when you customize the classifier, you will be changing the first section, where the packet is analyzed.

7.3 How to Create a Custom Classifier

Follow these steps:

1. Make changes to the default classifier (`$TILERA_ROOT/src/sys/mpipe/classifier.c`) as needed.
2. Use `tile-mpipe-cc`, the C compiler for the classifier, to compile your customized version of the classifier. See [Section 7.4 “Using tile-mpipe-cc to Compile a Custom Classifier” on page 38](#).
3. Include the custom classifier in bootroms, using either of these methods:
 - Use the `--classifier` option to `tile-monitor` when booting or creating a bootrom.
 - Use `gxio_mpipe_classifier_load_from_file()` to dynamically load it.

7.4 Using tile-mpipe-cc to Compile a Custom Classifier

`tile-mpipe-cc` is the C compiler for the classifier processor. You can use it to create a custom version of the default `classifier.c`.

For details about what `tile-mpipe-cc` supports and what it does not support, see the `tile-mpipe-cc.html` file in the MDE Document Index.

`tile-mpipe-cc` has no way to allow special handling of exceptions, so the default behavior is to drop a packet in packet processing when an exception occurs.

7.5 Using printf() in the Classifier

If you want to use `printf()` in the classifier, you must customize the `classifier.c` program and run your application under the simulator.

To use `printf()`, do the following:

1. Delete this block:


```
#ifndef SPEWING
#define printf(...) /*NOTHING*/
#endif
```
2. Delete any `printf()` in `classifier.c` that you do not want.
3. Add `printf()` commands specific to your debugging needs.
4. Run the simulator with a very slow bit rate, thus to prevent output interleaving and to ensure that only one packet is handled at a time.

CHAPTER 8 MANAGING NETWORK LINKS

This chapter describes mPIPE physical network link management:

- [Section 8.1 Understanding mPIPE Link Management](#)
- [Section 8.2 Regulating mPIPE Link Permissions](#)
- [Section 8.3 Data Structures for Link Management](#)
- [Section 8.4 Naming Links](#)

Note: This chapter provides an introduction to the link management API. For more information, see the “mPIPE Link Management” section of the *Application Libraries Reference Manual* (UG527).

8.1 Understanding mPIPE Link Management

Tilera provides a link management API for sending and manipulating the state and configuration of physical network links.

The mPIPE link management model revolves around three different states, which are maintained for each link:

- The current link state: Is the link up now, and if it is, at what speed?
- The desired link state: What should the link state be?

The system is always working to make the desired state the current state. If the desired state is up, and the link is down, the system is constantly trying to bring it up automatically.

- The possible link state: What speeds are valid for this particular link? Or, in other words, what are the capabilities of the link hardware?

8.2 Regulating mPIPE Link Permissions

As a security feature, mPIPE provides link permissions. Link permissions control what can be done with a link, and potentially what permissions can be granted to other processes.

There are different kinds of permissions:

- Data permission

Allows a process to receive packets from the link by specifying the link’s channel number in mPIPE packet distribution rules, and to send packets to the link by using the link’s channel number as the target for an eDMA ring.

- Stats permission

Allows a process to retrieve link attributes (such as the speeds it is capable of running at, or whether it is currently up), and to read and write certain statistics-related registers in the link’s MAC.

- Control permission

Allows a process to retrieve and modify link attributes (so that it can, for example, bring the link up and take it down), and read and write many registers in the link's MAC and PHY.

Any permission can be requested as shared or exclusive.

Shared mode allows other processes to also request shared permission.

Exclusive mode prevents other processes from requesting it.

In keeping with GXIO's typical usage in an embedded environment, the defaults for all permissions are shared.

Permissions are granted on a first-come, first-served basis. Thus, if two applications request an exclusive permission on the same link, the one to run first will win. However, some system components, like the kernel Ethernet driver, might get an opportunity to open links before any applications run.

8.3 Data Structures for Link Management

These are the data structures that applications can use for managing physical network links:

- `gxio_mpipe_link_t` (an object used to manage mPIPE link state and resources)
- `gxio_mpipe_stats_t` (mPIPE statistics structure)

8.4 Naming Links

Link names can be in the following forms:

- *gbenumber* (for Gigabit Ethernet)
- *xgbenumber* (for 10 Gigabit Ethernet),
- *loopnumber* (for internal mPIPE loopback)
- *ilknumber/channel* (for Interlaken links)

Examples of possible link names are: *gbe0*, *xgbe1*, *loop3*, and *ilk0/12*.

The maximum number of characters allowed in a link name is 32.

The correspondence between the link name and an mPIPE instance number or mPIPE channel number is system-dependent.

Not all links will exist on all systems.

The set of numbers used for a particular link type might not start at zero and might not be contiguous.

To retrieve the set of links that exist on a system, use `gxio_mpipe_link_enumerate()`.

To determine which mPIPE controls a particular link, use `gxio_mpipe_link_instance()`.

An application can use multiple links.

CHAPTER 9 PROGRAMMING MULTIPLE mPIPE INSTANCES

This chapter describes how to program to support multiple mPIPE instances:

- [Section 9.1 GXIO API Changes for TILE-Gx72](#)
- [Section 9.2 Programming for Multiple mPIPE Instances](#)

9.1 GXIO API Changes for TILE-Gx72

In order to use the multiple mPIPE engines that the new TILE-Gx72 chip provides, we enhanced the GXIO API to support two instances of mPIPE.

Note: This multiple mPIPE instance architecture is an interim API, and may change in the final version.

The TILE-Gx72 does not include hardware support for remote buffer return across the two instances of mPIPE. The GXIO API provides an operation mode which can perform remote buffer return operations which are transparent to the user. To use this mechanism, your application needs to copy the ingress descriptor's buffer descriptor into the egress descriptor and set the `hwb` bit; After it egresses the packet, the GXIO API performs the remote buffer return internally if the egress buffer belongs to the other instance of mPIPE. To enable this multiple mPIPE operation mode, define the macro `GXIO_MULTI_PIPES` in the Makefile or in the header file.

Note: In order to support multiple mPIPE mode, we changed the default Classifier to encode the mPIPE instance ID in the ingress packet descriptor. The Classifier uses the upper bit (bit 7) of the Byte 7 (CTR0 is defined in its lower 5 bits) in the 64 byte packet descriptor as one bit instance ID. All packet descriptors generated by mPIPE instance 0 have the instance ID set to 0 while the packet descriptors came from mPIPE instance 1 have instance ID as 1. If you have made use of the same reserved bit (bit 7 of Byte 7) in the packet descriptor for some other purpose in an Application, the GXIO library will not work properly in multiple mPIPE mode.

To support a multiple mPIPE architecture, we changed the GXIO library with respect to:

- mPIPE context initialization
- Buffer stack allocation routines
- Memory registration routines

9.1.1 Configuring mPIPE Instance Mode

To configure support for multiple mPIPE instance mode, we provide these new compilation flags:

```
GXIO_MULTI_PIPES
GXIO_RHWB_NONE
```

To enable multiple mPIPE instance mode with remote mPIPE buffer return support, specify the `GXIO_MULTI_PIPES` compilation flag.

To disable the remote mPIPE buffer return support for multiple mPIPE instance mode, specify the `GXIO_RHWP_NONE` compilation flags.

You may specify these compilation flags either in the header file `mpipe.h` or in the `gcc CFLAGS` option `-DGXIO_MULTI_MPIPES` in your Makefile.

Note: To use multiple mPIPE mode, you must compile all modules with `GXIO_MULTI_PIPES` defined in order to avoid any inconsistency.

9.1.2 Initializing Multiple mPIPE Instance Mode

You must initialize mPIPE to use multiple mPIPE instance mode by passing `NULL` as the context pointer through the `gxio_mpipe_init()` routine. It is legal to perform this initialization more than once for a particular mPIPE.

After calling the `gxio_mpipe_init()` routine, use the `GXIO` macro `GXIO_MPIPE_CONTEXT(instance_number)` for each instance's mPIPE context pointer. The following code shows how to initialize two mPIPE instances:

```
...
// get the instance id from the link name.
int instance_1 = gxio_mpipe_link_instance("xgbe1");

// Initialize the gxio's mpipe contexts.
int retval = gxio_mpipe_init(NULL, instance_1);

// Check if return error.
if (retval != 0)
    exit(1);

// Save the mpipe context of link xgbe1 after the initialization.
gxio_mpipe_context_t *context_xgbe1 = GXIO_MPIPE_CONTEXT(instance_1);

// Get the instance id from the link name.
int instance_2 = gxio_mpipe_link_instance("xgbe5");

// Initialize the gxio's mpipe contexts.
retval = gxio_mpipe_init(NULL, instance_2);

// Check if return error.
if (retval != 0)
    exit(1);

// Save the mpipe context of link xgbe5 after the initialization.
gxio_mpipe_context_t *context_xgbe5 = GXIO_MPIPE_CONTEXT(instance_2);
...
```

9.1.3 Manipulating mPIPE Buffer Descriptors

To track the packet buffer, the `GXIO` API adds the buffer descriptor data structure called `gxio_mpipe_bdesc_t`, which contains buffer starting address, mPIPE instance ID and buffer stack ID.

To extract the buffer descriptor of an ingress packet from its ingress descriptor, use the `gxio_mpipe_idesc_to_bdesc()` routine, as in the following example.

```
/** Extract buffer descriptor from some fields in an "idesc". */
gxio_mpipe_bdesc_t gxio_mpipe_idesc_to_bdesc(gxio_mpipe_idesc_t *idesc);
```

To set the buffer descriptor into an egress descriptor before egressing the packet, use the `gxio_mpipe_edesc_set_bdesc()` routine, as in this example:

```
/** Set buffer info. of an egress descriptor based on a buffer descriptor. */
void gxio_mpipe_edesc_set_bdesc(
    gxio_mpipe_edesc_t* edesc,
    gxio_mpipe_bdesc_t bdesc);
```

9.1.4 Allocating Buffer Stacks for Multiple mPIPE Instances

The `gxio_mpipe_alloc_buffer_stacks()` routine automatically buffers stacks from both mPIPE instances, as in this example:

```
/** Allocate a set of buffer stacks. */
extern int
gxio_mpipe_alloc_buffer_stacks(
    gxio_mpipe_context_t* context,
    unsigned int count, unsigned int first,
    unsigned int flags);
```

Note: If there is only one instance, allocate buffers only from that instance.

9.1.5 Registering Buffers for Multiple mPIPE Instances

When in multiple mPIPE instance mode, the GXIO library registers a page to both mPIPE instance's IOTLB, regardless which context you provide, as in the following example:

```
/** Register a set of buffer stacks. */
extern int
gxio_mpipe_register_page(gxio_mpipe_context_t* context,
    unsigned int stack,
    void* page, size_t page_size,
    unsigned int page_flags);
```

Note: The `gxio_mpipe_register_page()` routine registers to a single mPIPE if there is only one instance of mPIPE.

9.2 Programming for Multiple mPIPE Instances

This is an example to forward packets from one link to another link.

The code examples which follow are all taken from the file `$TILERA_ROOT/example/mpipe/forward/app_gx8072.c`.

1. Parse the ingress and egress link names, as in this example:

```
{
    int result;
    int instance;

    char* link_name_in = "gbe0";
    char* link_name_out = "gbe16";

    // Parse args.
    for (int i = 1; i < argc; i++)
    {
        char* arg = argv[i];
```

```

if (!strcmp(arg, "--link") && i + 1 < argc)
{
    // --link <link_name_in> <link_name_out>
    link_name_in = argv[++i];
    if (strstr(argv[i + 1], "gbe"))
    {
        link_name_out = argv[++i];
    }
}

```

Associate the mPIPE context with ingress and egress links, as in this example:

```

enum {
    // Ingress link.
    LINK_INGRESS = 0,
    // Egress link.
    LINK_EGRESS,
    // Total Link count.
    LINK_MAX
};

// The mpipe contexts of links, initialized as all NULL.
static gxio_mpipe_context_t* context[LINK_MAX] =
{ NULL };

```

2. Get the ingress mPIPE instance, as in this example:

```

// Get the instance.
instance = gxio_mpipe_link_instance(link_name_in);
if (instance < 0)
    tmc_task_die("Link '%s' does not exist.", link_name_in);

```

3. Get the mPIPE context of the ingress link, as with this example:

```

// Get the gxio mPIPE context of the instance just initialized.
context[LINK_INGRESS] = GXIO_MPIPE_CONTEXT(instance);

gxio_mpipe_link_t link;
result = gxio_mpipe_link_open(&link,
                             context[LINK_INGRESS],
                             link_name_in,
                             0);
VERIFY(result, "gxio_mpipe_link_open()");

```

4. Use the ingress link's contexts when you allocate NotifRings, as in this example:

```

// Allocate some NotifRings.
result = gxio_mpipe_alloc_notif_rings(context[LINK_INGRESS],
                                     num_workers,
                                     0,
                                     0);
VERIFY(result, "gxio_mpipe_alloc_notif_rings()");
unsigned int ring = result;

```

5. Use the ingress link's contexts when you initialize the NotifRings, as in this example:

```

// Init the NotifRings.
size_t notif_ring_entries = 512;
size_t notif_ring_size = notif_ring_entries * sizeof(gxio_mpipe_idesc_t);
size_t needed = notif_ring_size + sizeof(gxio_mpipe_iqueue_t);
for (int i = 0; i < num_workers; i++)
{
    tmc_alloc_t alloc = TMC_ALLOC_INIT;
    tmc_alloc_set_home(&alloc, tmc_cpus_find_nth_cpu(&cpus, i));
    // The ring must use physically contiguous memory, but the iqueue
    // can span pages, so we use "notif_ring_size", not "needed".

```

```

tmc_alloc_set_pagesize(&alloc, notif_ring_size);
void* iqueue_mem = tmc_alloc_map(&alloc, needed);
if (iqueue_mem == NULL)
    tmc_task_die("Failure in 'tmc_alloc_map()'");
gxio_mpipe_iqueue_t* iqueue = iqueue_mem + notif_ring_size;
result = gxio_mpipe_iqueue_init(iqueue,
                                context[LINK_INGRESS],
                                ring + i,
                                iqueue_mem,
                                notif_ring_size,
                                0);
VERIFY(result, "gxio_mpipe_iqueue_init()");
iqueues[i] = iqueue;
}

```

6. Use the ingress link's contexts when you allocate a NotifGroup, as in this example:

```

// Allocate a NotifGroup.
result = gxio_mpipe_alloc_notif_groups(context[LINK_INGRESS], 1, 0, 0);
VERIFY(result, "gxio_mpipe_alloc_notif_groups()");
int group = result;

```

7. Allocate memory buckets by supplying the ingress mPIPE context to the `gxio_mpipe_alloc_buckets()` routine, as in this example:

```

// Allocate some buckets. The default mPipe classifier requires
// the number of buckets to be a power of two (maximum of 4096).
int num_buckets = 1024;
result = gxio_mpipe_alloc_buckets(context[LINK_INGRESS],
                                   num_buckets,
                                   0,
                                   0);
VERIFY(result, "gxio_mpipe_alloc_buckets()");
int bucket = result;

```

8. Call the `gxio_mpipe_init_notif_group_and_buckets()` routine with the ingress mPIPE context pointer, as in this example:

```

// Init group and buckets, preserving packet order among flows.
gxio_mpipe_bucket_mode_t mode = GXIO_MPIPE_BUCKET_DYNAMIC_FLOW_AFFINITY;
result = gxio_mpipe_init_notif_group_and_buckets(context[LINK_INGRESS],
                                                  group,
                                                  ring,
                                                  num_workers,
                                                  bucket,
                                                  num_buckets,
                                                  mode);
VERIFY(result, "gxio_mpipe_init_notif_group_and_buckets()");

```

9. Use the egress link's contexts when you initialize the Egress queue, as in this example:

```

// Initialize the equeue.
instance = gxio_mpipe_link_instance(link_name_out);
result = gxio_mpipe_init(NULL, instance);
VERIFY(result, "gxio_mpipe_init()");

context[LINK_EGRESS] = GXIO_MPIPE_CONTEXT(instance);
if (strcmp(link_name_in, link_name_out))
{
    result = gxio_mpipe_link_open(&link,
                                   context[LINK_EGRESS],
                                   link_name_out,
                                   0);
}

```

```

VERIFY(result, "gxio_mpipe_link_open()");

channel = gxio_mpipe_link_channel(&link);
if (jumbo)
{
    // Allow the link to receive jumbo packets.
    gxio_mpipe_link_set_attr(&link, GXIO_MPIPE_LINK_RECEIVE_JUMBO, 1);
}
}
result = gxio_mpipe_alloc_edma_rings(context[LINK_EGRESS], 1, 0, 0);
VERIFY(result, "gxio_mpipe_alloc_edma_rings");
uint ering = result;
size_t edescs_size = equeue_entries * sizeof(gxio_mpipe_edesc_t);
tmc_alloc_t edescs_alloc = TMC_ALLOC_INIT;
tmc_alloc_set_pagesize(&edescs_alloc, edescs_size);
void* edescs = tmc_alloc_map(&edescs_alloc, edescs_size);
if (edescs == NULL)
    tmc_task_die("Failed to allocate equeue memory.");
result = gxio_mpipe_equeue_init(equeue,
                                context[LINK_EGRESS],
                                ering,
                                channel,
                                edescs,
                                edescs_size,
                                0);
VERIFY(result, "gxio_mpipe_equeue_init()");

```

10. Allocate the buffer stack using the ingress mPIPE context in the `gxio_mpipe_alloc_buffer_stacks()` routine, as in this example:

```

// Use enough small/large buffers to avoid ever getting "idesc->be".
unsigned int num_bufs = equeue_entries + num_workers * notif_ring_entries;

// Allocate small/large/jumbo buffer stacks.
instance = gxio_mpipe_link_instance(link_name_in);
result = gxio_mpipe_alloc_buffer_stacks(context[LINK_INGRESS],
                                        jumbo ? 3 : 2,
                                        0,
                                        0);

VERIFY(result, "gxio_mpipe_alloc_buffer_stacks()");
int stack_idx = result;

```

11. Use the ingress mPIPE context in creating small and large stacks, as in this example:

```

// Initialize small/large stacks.
create_stack(context[LINK_INGRESS], stack_idx + 0,
             GXIO_MPIPE_BUFFER_SIZE_128, num_bufs);
create_stack(context[LINK_INGRESS], stack_idx + 1,
             GXIO_MPIPE_BUFFER_SIZE_1664, num_bufs);

```

12. Use the ingress mPIPE context when you initialize and start the mPIPE rules engine, as in this example:

```

// Register for packets.
gxio_mpipe_rules_t rules;
gxio_mpipe_rules_init(&rules, context[LINK_INGRESS]);
gxio_mpipe_rules_begin(&rules, bucket, num_buckets, NULL);
result = gxio_mpipe_rules_commit(&rules);
VERIFY(result, "gxio_mpipe_rules_commit()");

```

CHAPTER 10 FREQUENTLY ASKED QUESTIONS (FAQs)

This chapter includes frequently asked questions concerning mPIPE.

10.1 Basic FAQs

What is the difference between the `size` and `xfer_size` fields of `gxio_mpipe_edesc_t`?

`size` is the “kind” of buffer (that is, 128, 256, 1518, and so forth). `xfer_size` is the actual byte count.

What is the `bound` field of `gxio_mpipe_edesc_t`?

Whether or not the `edesc` is the “last” piece of a packet.

Can the mode of the load balancer be dynamically changed?

Yes. Each bucket (including its mode) can be re-initialized.

Do we need IOTLB (I/O translation lookaside buffer) entries for anything besides the buffers?

No.

The memory for the buffer stack, ingress ring, and egress ring is “pinned.” The `iqueue` and `equeue` structures are not pinned, as they are purely user-level structures.

I got an error code of -22. What does this mean?

This is `-EINVAL`. Your value was invalid.

In the `gxio_mpipe_iqueue_init()` routine, what can be passed to `mem_flags`?

This is currently unspecified.

What happens to the contents of the egress packet descriptor?

The contents of the egress packet descriptor, `gxio_mpipe_edesc_t`, are copied into the egress DMA (eDMA) ring. Note: The eDMA ring does not simply keep a pointer to the egress packet descriptor.

What is the correspondence of channels to physical ports?

Currently, the following:

- 0 - 15 = GbE 0 - 15
- 0/4/8/12 = XGbE 0 - 4
- 16 - 19 = loopback

10.2 FAQs About How to Perform Tasks

How do I specify that a tile should receive or transmit a packet from a particular channel (port)?

Use `gxio_mpipe_rules_add_channel()` to create a new rule.

What happens if I release a buffer into a different stack?

Results are undefined.

I have an application that uses flow hashing. Can I use just a single bucket?

Yes. It is not a good proposal to use a single bucket for flow hash. If you configure the load balancer for FIXED (`STATIC_FLOW_AFFINITY`) mode, a bucket is statically assigned to a specific NotifRing. Even with flow hashing, all traffic go to only one ring and worker.

When I use the `gxio_mpipe_iqueue_try_peek()` routine and get a return value greater than 1, how do I access all the entries and the consume them? Do they have to be consumed in order?

The ingress descriptor entries (`gxio_mipe_idesc_t`) must be accessed and consumed in order. You cannot access or consume more than the value returned by `gxio_mpipe_iqueue_peek()`. But you can access and consume less than the return value.

How do I get a channel number to use for creating classifier rules and setting up eDMA rings?

Use the `gxio_mpipe_link_channel()` routine.

How do I specify a particular channel or VLAN or destination MAC?

For information about how to do this, see the example described in [Section 3.9 “Multi-Rules Example”](#) on page 20.

G GLOSSARY

Acronyms

Acronym	Definition
AAD	Additional Authenticated Data.
AES	Advanced Encryption Standard.
AH	Authentication Header. For use with IPsec.
AHB	Advanced High-performance Bus.
AIC	Advanced Interrupt Controller.
ARC4	"Alleged RC4", compatible with RC4.
AXI	Advanced eXtensible Interface.
BARs	Base address registers.
BIST	Built in Self Test.
bucket	Used by the load balancer to determine which application will receive the packet.
buffer chaining	A method of supporting automatic scatter for ingress. With buffer chaining, when a packet exceeds the size of a single buffer, the packet is fragmented across multiple buffers and a link is created from each buffer to the subsequent buffer.
CAM	Content Addressable Memory.
CBC	Cipher Block Chaining. A mode of operation for block cipher encryption.
CDR	Clock Data Recovery. Indicates when a signal is sent without a clock. Instead, the clock is extracted from the datastream.
CFB	Cipher Feed Back. A mode of operation for block cipher encryption.
channel	A set of physical resources in the mPIPE. Each MAC is associated with one or more channels. Depending on system configuration, a channel might be shared by more than one MAC.
classifier	Hardware: A processor that parses incoming packets and determines which flow the packet belongs to. Software: The binary program that implements the parsing and packet processing.
CPLD	Complex PLD.
CPLH	Completion Header.
CRT	Chinese Remainders Theorem.
CTR	Counter Mode. A mode of operation for block cipher encryption.
DES	Data Encryption Standard.
DESP	Descriptor Pointer.

Acronym	Definition
DF	Don't Fragment, this is a bit in the IPv4 header.
DH	Diffie-Hellman.
DLL	Data Link Layer.
DLLP	Data Link Layer Packet.
DMA	Direct Memory Access.
DSA	Digital Signature Algorithm.
DST	Destination.
ECB	Electronic Code Book. A mode of operation for block cipher encryption.
ECC	Error Correcting Code.
ECDH	Elliptic Curve (version of) Diffie-Hellman.
ECDSA	Elliptic Curve Digital Signature Algorithm.
ECN	Explicit Congestion Notification .
ede	Encryption-decryption-encryption.
edesc	Egress packet descriptor, <code>gxio_mpipe_edesc_t</code> . Pointer to <code>MPIPE_EDMA_DESC_t</code> .
eDMA ring	Egress direct memory access ring. A descriptor ring. Each eDMA ring is associated with one channel and one MAC.
egress DMA	Egress direct memory access.
equeue	A data structure wrapped around an eDMA ring, with functions such as <code>gxio_mpipe_equeue_put()</code> .
ESP	Encapsulated Security Payload.
FC	Flow Control.
FIFO	First In First Out.
FIPS	Federal Information Processing Standard.
GB	Gigabyte.
Gbit	Gigabit.
Gbps	Gigabits per second.
GCD	Greatest Common Divisor.
GCM	Galois Counter Mode. A mode of operation for block cipher encryption.
HA	High Assurance.
hash bucket	A bucket computed from the hashed flowID of a packet.
HFH	Hash-for-Home.
HMAC	Hash Message Authentication Code. A special kind of keyed hash.
HPI	See host port interface.
HSM	Hardware Security Module.
HW	Hardware.
IANA	Internet Assigned Number Authority.

Acronym	Definition
ICM	Integer counter mode (as in AES-ICM).
ICMP	Internet Control Message Protocol.
ICV	Integrity Check Value.
idesc	Ingress packet descriptor, <code>gxio_mpipe_idesc_t</code> . Pointer to <code>MPIPE_PDESC_t</code> .
iDMA	Ingress direct memory access.
IDN	I/O Dynamic Network.
IETF	Internet Engineering Task Force.
IHL	Internet Header Length.
IKE	Internet Key Exchange. Used in IPsec to establish a Security Association (SA)
ingress packet descriptor	A 64-byte summary of the classification, load balancing, and buffer management aspects of ingress processing. The classifier creates the packet descriptor for each incoming packet. Known in software as an <code>idesc</code> or <code>PDESC</code> .
IP	Internet Protocol/ Intellectual Property.
IPI	Interprocessor Interrupt.
IPv4	Internet Protocol version 4.
IPv6	Internet Protocol version 6.
iqueue	A data structure wrapped around a notification ring, with functions such as <code>gxio_mpipe_iqueue_get()</code> .
ITCM	Instruction Tightly Coupled Memory.
IV	Initialization Vector. Used in cryptography to begin a block cipher mode of operation.
KB	Kilo byte (1024 Bytes).
Kbit	Kilo bit (1024 bits).
Kbps	Kilo bit per second.
KDK	Key Decryption Key.
KEK	Key Encryption Key.
LNME	Large Number Multiplier and Exponentiator.
load balancer	A component of mPIPE that assigns incoming packets to notification rings.
LSB	Least Significant Bit.
LSW	Least Significant Double word (dword).
LTSSM	Link Training and Status State Machine.
MAC	Media Access Control data communication protocol sublayer. A physical device connected to the mPIPE. Can include more than one channel. Also known as a port.
MAC	Message Authentication Code.
MB	Megabyte (1024 KB).
Mbit	Mega bit (1024 Kbit).

Acronym	Definition
Mbps	Mega bits per second.
MD5, MD-5	Message-Digest algorithm 5.
MDE™	Multicore programming environment.
MKI field	Master Key Information field.
MMIO	Memory-mapped I/O.
mPIPE context object	An object used to manage mPIPE hardware resources. Also known as mPIPE context.
mPIPE™	Multicore Programmable Intelligent Packet Engine.
MPL	Minimum Protection Level.
MPS	Maximum packet size. The MPS bitfield is in the TRIO_MAC_CONFIG register.
MRS	Maximum request size. The MRS bitfield is in the TRIO_MAC_CONFIG register.
MSB	Most Significant Bit.
MSb/MSbs	Most Significant Bit (plural: MSbs).
MSB/MSBs	Most Significant Byte (plural: MSBs).
MSW	Most Significant Double Word (dword).
MTU	Maximum Transfer Unit.
NAT	Network Address Translation.
NAT-T	NAT-Traversal.
NH	Next Header.
notification group	Also known as NotifGroup. A notification group is composed of a set of notification rings. Notification groups are a feature to support multiple load-balancing domains.
notification ring	Also known as NotifRing. A data structure stored in Tile memory containing ingress packet descriptors created by the classifier associated with a single tile.
NPD	Nonposted Data.
NPH	Nonposted Header.
NVRAM	Non Volatile RAM.
ODT	On-Die Termination.
OFB	Output Feed Back. A mode of operation for block cipher encryption.
OTP	One Time Programmable.
packet descriptor	See ingress packet descriptor.
PCS	Physical Coding Sublayer.
PD	Posted Data.
PD	Pull-Down (internal resistor on input pads forcing logic '0' input when not connected).
PH	Posted Header.
PIO	Programmed input/output.
PKA	Public Key Accelerator.

Acronym	Definition
PKCP	Public Key Co-Processor.
PKE	Public Key Exchange.
PKI	Public Key Infrastructure.
PRNG	Pseudo Random Number Generator.
RAM	Random Access Memory.
RDN	Response Dynamic Network.
RGMI	Reduced Gigabit Media Independent Interface.
RNG	Random Number Generator.
ROM	Read Only Memory.
RSA	Public key encryption, private key decryption.
RTP	Real-time Transport Protocol.
SA	Security Association.
SDN	Shared Dynamic Network.
SHA	Secure Hash Algorithm.
SOC	System On (a) Chip.
SPI	Security Parameter Index.
SPI-SROM	Serial Flash with serial peripheral interface. Also known as SROM.
SPRAM	Single Port Random Access Memory.
SQ	Scatter Queue.
SRAM	Static Random Access Memory.
SRC	Source.
SRTP	Secure RTP.
SSL	Secure Sockets Layer. Related to TLS.)
STI	MACsec SecTag Information.
SWDS Port	Switch Downstream Port.
TC	Traffic Class.
TCM	Tightly Coupled Memory (memory interface protocol).
TCP	Transmission Control Protocol.
TFC	Traffic Flow Confidentiality.
TL	Transaction Layer.
TLB	Translation Lookaside Buffer.
TLP	Transaction Layer Packet.
TLS	Transport Layer Security, related to SSL.
TOS	Type Of Service.
TRNG	True Random Number Generator (based on a physical noise source).

Acronym	Definition
TTL	Time To Live.
UART	Universal Asynchronous Receiver Transmitter.
UDN	User Dynamic Network.
UDP	User Datagram Protocol.
UL	User Logic.
VC	Virtual Channel.
VLAN	Virtual local area network. This is an optional field in the L2 packet header.
VPP	Verilog Pre-processor.
WEP	Wired Equivalent Privacy

Glossary

Term	Definition
AH	Authentication Header, an IPsec mode that authenticates the entire IP packet.
anti-replay	An IPsec security feature that prevents a third party from eavesdropping on an IPsec conversation.
black data	Data that is present on the public domain, it is typically encrypted data.
Block	In the Key Wrap context, this is a 64-bit wide chunk of data.
Ciphertext	The encrypted data / wrapped key.
Channel	A set of physical resources in the mPIPE. Each MAC is associated with one or more channels. Depending on system configuration, more than one MAC might share a channel. For example, when the two MACs share a set of input pins and hence can't be in service simultaneously.
Classifier	Processor that parses incoming packets and determines to which flow the packet belongs.
clock stretching	The I²C bus provides an explicit clock signal that relieves master and slave from synchronizing exactly to a predefined baud rate. In some cases an I²C slave is not able to co-operate with the clock speed given by the master and needs to slow down a little. This delay is introduced by a mechanism referred to as <i>clock stretching</i>.
Context	Hardware entity that contains information (interrupt bindings, state, source and destination data pointers, opcodes, and other engine-specific information) for processing data. The MiCA processes data associated with one Context at a time. MiCA instances typically have 40 Contexts, which are independent of each other. Under typical operation, a Context is allocated to a particular tile and that tile instructs the operation of the Context.
Context Record	A data structure used with Tiler Crypto Packet Processor engine that contains processing parameters and keying data.
CPLD	Complex PLD. A programmable logic device (PLD) that is made up of several simple PLDs (SPLDs) with a programmable switching matrix in between the logic blocks. CPLDs typically use EEPROM, flash memory or SRAM to hold the logic design interconnections.

Term	Definition
CTR	Counter mode of operation for block cipher algorithms (as in AES-CTR).
dword	A 32-bit data word.
Flit	One pump of data.
Key	KEK.
Key Data	Either a (plaintext) key that needs to be wrapper or a wrapped key (ciphertext) that needs to be unwrapped.
ECB	Electronic Code Book.
ECC	Elliptic Curve Cryptography.
ECC	Error-Correcting Code. A type of memory that automatically detects and corrects errors.
ECN	Explicit Congestion Notification. Notification of congestion on the route in one or more nodes when a packet travels on a network from host A to host B.
eDMA Ring	An egress descriptor ring. Each ring is associated with exactly one channel and hence exactly one MAC/Port.
ESP	Encapsulated Security Payload, an IPsec mode that crypts the data after the IPsec header.
Fast-mode and Fast-Mode Plus	Refer to I²C modes on page 55 .
HOP	A node in an IPv6 network.
HotPlug	A system for managing devices that can be dynamically attached to and removed from the system while it is running. The most obvious use for this system is handling USB and firewire devices, though it also handles PCI (32-bit PCMCIA - or CardBus - devices are really PCI in disguise), tape drives, SCSI devices, devices requiring firmware to be loaded into them, input devices, and more.
host port interfaces (HPIs)	A 16-bit-wide parallel port through which a host processor can directly access the CPU's memory space. The host device functions as a master to the interface, which increases ease of access. The host and CPU can exchange information via internal or external memory. The host also has direct access to memory-mapped peripherals. Connectivity to the CPU's memory space is provided through the DMA controller.
Hypervisor Services	Provided to support two basic operations: install a new page table (performed on context switch), and flush the TLB (performed after invalidating or changing a page table entry). On a page fault, the client receives an interrupt, and is responsible for taking appropriate action (such as making the necessary data available via appropriate changes to the page table, or terminating a user program which has used an invalid address).
I ² C modes	I²C modes include: standard-mode, fast-mode and fast-mode plus. - Standard-mode refers to the initial transfer speed mode of the I²C specification which allows up to 100 kbit/s. - The fast-mode features 400 kbit/s, fast-mode plus up to 1000 kbit/s, while the high speed HS-mode runs with up to 3.4 Mbit/s. For more information refer to http://www.i2c-bus.org.
inbound data	Packet data (typically encrypted) entering the Crypto Packet Processor from the public domain. This data is typically decrypted by the Crypto Packet Processor.

Term	Definition
Interpacket Gap (IPG)	Ethernet devices must allow a minimum idle period between transmission of Ethernet frames known as the interframe gap (IFG) or interpacket gap (IPG). It provides a brief recovery time between frames to allow devices to prepare for reception of the next frame.
Input Token	A data structure that provides the Crypto Packet Processor with packet processing instructions. Basic input tokens are provided by Tileria and each performs a particular operation. A copy of the input token must be provided with each packet to be processed, with its parameter fields tailored to the particular packet.
IPSec	IP Security Protocol, supports two modes ESP and AH.
KASUMI f8	3GPP confidentiality algorithm.
KASUMI f9	3GPP integrity algorithm.
Load Balancer	Assigns incoming packets to NotifRings.
MAC	A physical device connected to the mPIPE. This device might include more than one channel.
MDIO	Management interface I/O bidirectional pin. The management interface controls the behavior of the PHY.
MiCA	Multicore iMesh Coprocessing Accelerator (MiCA™). The MiCA provides a common front-end (both software and hardware) to various I/O off-load or acceleration functions, for example Crypto or Compression. The MiCA performs operations as specified by an opcode on data in memory.
MMIO	Memory mapping region. Communication with the mPIPE uses the MMIO interface. The physical address is broken into fields that map the address into the various mPIPE regions.
MMIO Registers	Tile access and control of the operation of the MiCA is provided via a set of memory mapped registers.
mPIPE	Multicore Programmable Intelligent Packet Engine. Engine that provides line rate services for the packet interfaces. These services are also available for packet data stored in on-chip buffers, such as flows being moved to and from a PCI interface.
NotifRing	Data structure stored in Tile memory containing ingress packet descriptors created by the classifier.
Operation	A complete wrap or unwrap.
outbound data	Packet data (plaintext) entering the Crypto Packet Processor for the private domain. This data is typically encrypted by the Crypto Packet Processor.
PIO interface	Interface that provides direct PCI memory-space communication with the PCI port(s).
PIPE	PHY Interface for PCI Express.
Plaintext	The plain key data that needs to be wrapped or the result of an unwrap.
PLD	Programmable Logic Device is an electronic component used to build reconfigurable digital circuits
Port	Same as MAC.
Pseudo Random Number Generator	Devices or algorithms that output statistically independent and unbiased numbers. Also referred to as PRNG.

Term	Definition
Priority Queue	A set of virtual resources in the mPIPE. Ingress packets are assigned to a priority queue by the MAC. Egress packets are queued based on the configuration of their associated eDMA ring.
Promiscuous Mode	In computing, refers to a configuration of a network card wherein a setting is enabled so that the card passes all traffic it receives to the CPU rather than just packets addressed to it, a feature normally used for packet sniffing. Many operating systems require superuser privileges to enable promiscuous mode. A non-routing node in promiscuous mode can generally only monitor traffic to and from other nodes within the same collision domain (for Ethernet and Wireless LAN) or ring (for Token ring or FDDI).
RC4	The most widely used software stream cipher that is used in popular protocols, such as SSL (to protect Internet traffic) and WEP (to secure wireless networks).
red data	Data that is present on the private domain, this is typically plaintext.
QDN mesh interface	QDN is defined as the reQuest Dynamic Network. The Gx processor's mesh interface that enables MMIO accesses, effectively, Tiles read and write entries in the TLB.
Result Token	A data structure provided to the Crypto Packet Processor in which result status information from the operation is stored.
SA	Security Association. Used in IPSec context. Refers to the data structure describing the IPSec tunnel parameters (encryption/hash keys, transform choice, sequence numbers, etc.)
Scatter Queue	Scatter queue (SQ) regions provide a means for mailbox/doorbell communication via PCI Express. Each scatter region consists of a descriptor FIFO used to provide target VAs for incoming PCIe requests. Both reads and writes are supported to SQ regions. The TILE-Gx36™ processor provides eight SQ Regions.
SRIO	Serial RapidIO. Like RapidIO, it is a high-performance packet-switched, interconnect technology for interconnecting chips on a circuit board, and also circuit boards to each other using a backplane. Serial RapidIO differs from RapidIO, which has a separate clock signal, in that it uses clock recovery in the receiver to generate a clock from the data stream.
Standard-mode	Refer to I²C modes on page 55 .
Tag	Refer to the TRIO_TILE_PIO_CTL register.
Text	General term representing plaintext and/or ciphertext.
token	Packet specific data structure containing the information that the Crypto Packet Processor needs to process a packet.
TRIO	Tile Programmed I/O (PIO).
UART	(Universal Asynchronous Receiver Transmitter). The electronic circuit that makes up the serial port. Also known as "universal serial asynchronous receiver transmitter" (USART), it converts parallel bytes from the CPU into serial bits for transmission, and vice versa. It generates and strips the start and stop bits appended to each character.
VLIW architecture	VLIW (Very Long Instruction Word). A microprocessor design technology. A chip with VLIW technology is capable of executing many operations within one clock cycle. Essentially, a compiler reduces program instructions into basic operations that the processor can perform simultaneously. The operations are put into a very long instruction word that the processor then takes apart and passes the operations off to the appropriate devices.

INDEX

A

AAD
 defined [49](#)
AES
 defined [49](#)
AH
 defined [54](#)
anti-replay
 defined [54](#)
audience of this guide [vii](#)
AXI
 defined [49](#)

B

BARs
 defined [49](#)
BIST
 defined [49](#)
black data
 defined [54](#)
block
 defined [54](#)
bucket
 distributing packets with a load balancer [3](#)

C

calloc() [11, 13, 22, 23](#)
CAM
 defined [49](#)
CBC
 defined [49](#)
CDR [49](#)
 defined [49](#)
CFB
 defined [49](#)
channel
 defined [54](#)
ciphertext
 defined [54](#)
classifier [25, 39, 41](#)
 defined [54](#)
Clock Data Recovery
 See CDR
clock stretching
 defined [54](#)
context

 defined [54](#)
 record
 defined [54](#)
conventions
 notation [ix](#)
CPLD
 defined [49, 54](#)
CPLH
 defined [49](#)
CTR
 defined [55](#)
customer support [viii](#)

D

DESP
 defined [49](#)
distributing
 packets [3](#)
DLL
 defined [50](#)
DLLP
 defined [50](#)
document organization [vii](#)
dword
 defined [55](#)

E

ECB [50](#)
 defined [55](#)
ECC [50](#)
 defined [55](#)
ECDH
 defined [50](#)
ECDSA
 defined [50](#)
eDMA [3](#)
 ring
 defined [55](#)
eDMA ring [12, 14, 16, 17, 18, 20](#)
egress DMA ring [12, 14, 18](#)

F

fast-mode
 defined [55](#)
fast-mode plus
 defined [55](#)

- FC
 - defined [50](#)
- FG
 - defined [56](#)
- FIPS
 - defined [50](#)
- flit
 - defined [55](#)
- G**
- GCM
 - defined [50](#)
- gxio/mpipe.h [3](#)
- gxio_mpipe_alloc_buckets() [17](#)
- gxio_mpipe API [16, 18, 19](#)
- gxio_mpipe_alloc_buckets() [12, 13, 14, 18, 19, 20, 21, 23](#)
- gxio_mpipe_alloc_buffer_stacks() [12, 13, 14, 17, 18, 19, 20, 21](#)
- gxio_mpipe_alloc_edma_rings() [12, 13, 14, 17, 18, 20, 23](#)
- gxio_mpipe_alloc_notif_groups() [12, 13, 14, 17, 18, 19, 21, 22](#)
- gxio_mpipe_alloc_notif_rings() [11, 13, 14, 16, 18, 19, 20, 22](#)
- GXIO_MPIPE_BUCKET_ROUND_ROBIN [12, 18](#)
- gxio_mpipe_calc_buffer_stack_bytes() [12, 13, 15, 18, 21, 23](#)
- gxio_mpipe_equeue_init() [12, 13, 14, 17, 18, 20, 23](#)
- gxio_mpipe_equeue_put() [17](#)
- gxio_mpipe_get_sqn() [14, 23](#)
- gxio_mpipe_init() [11, 12, 14, 16, 18, 19, 20, 22](#)
- gxio_mpipe_init_buffer_stack() [12, 13, 15, 17, 18, 19, 20, 21, 23](#)
- gxio_mpipe_init_notif_group_and_buckets() [12, 13, 14, 17, 18, 19, 20, 21, 23](#)
- gxio_mpipe_init_notif_rings() [19](#)
- gxio_mpipe_iqueue_consume() [17](#)
- gxio_mpipe_iqueue_get() [19](#)
- gxio_mpipe_iqueue_init() [12, 13, 14, 16, 18, 19, 21, 22](#)
- gxio_mpipe_iqueue_peek() [17](#)
- gxio_mpipe_iqueue_try_peek() [17](#)
- gxio_mpipe_link_channel() [11, 13, 14, 20, 22](#)
- gxio_mpipe_link_instance() [11, 12, 14, 18, 20, 22](#)
- gxio_mpipe_link_open() [11, 12, 14, 19, 20, 22](#)
- gxio_mpipe_push_buffer() [12, 15, 17, 18, 19, 20, 21](#)
- gxio_mpipe_register_page() [12, 14, 15, 17, 18, 19, 20, 21, 23](#)
- gxio_mpipe_rules_add_channel() [12, 15](#)
- gxio_mpipe_rules_begin() [12, 14, 15, 17, 18, 20, 21, 23](#)
- gxio_mpipe_rules_commit() [15, 17, 18, 19, 21](#)
- gxio_mpipe_rules_init() [12, 14, 15, 17, 18, 20, 21, 23](#)
- gxio_mpipe_rules_set_headroom() [12, 15](#)
- H**
- HFH
 - defined [50](#)
- HMAC
 - defined [50](#)
- HOP
 - defined [55](#)
- host port interface
 - See* HPI
- HotPlug
 - defined [55](#)
- HPI [55](#)
 - defined [50, 55](#)
- HSM
 - defined [50](#)
- hugepages [21](#)
- Hypervisor services
 - defined [55](#)
- I**
- I2C
 - modes
 - defined [55](#)
- IANA
 - defined [50](#)
- ICM
 - defined [51](#)
- ICMP
 - defined [51](#)
- ICV
 - defined [51](#)
- IDE online help [viii](#)
- IDN
 - defined [51](#)
- IETF
 - defined [51](#)
- IHL
 - defined [51](#)
- inbound data
 - defined [55](#)
- info pages [viii](#)
- ingress queue [12, 13, 14, 18, 22](#)
- initializing
 - mPIPE driver [16](#)
- input token
 - defined [56](#)
- interframe gap
 - See* IFG
- Interpacket Gap
 - See* IPG
- IPG [56](#)
 - defined [56](#)
- IPI
 - defined [51](#)
- IPSec
 - defined [56](#)
- iqueue [12, 13, 14, 18, 22](#)
- ITCM
 - defined [51](#)
- IV [51](#)

K

KASUMI f8
 defined [56](#)
 KASUMI f9
 defined [56](#)
 KEK [51](#)
 key
 data
 defined [55](#)
 defined [55](#)

L

LNME
 defined [51](#)
 load balancer
 defined [56](#)
 LSB [51](#)
 LSW [51](#)
 LTSSM
 defined [51](#)

M

MAC
 defined [51](#), [56](#)
 man pages [viii](#)
 MD5, MD-5
 defined [52](#)
 MDE
 defined [52](#)
 MDE Document Index [viii](#)
 location of [viii](#)
 MDE documentation
 location of [viii](#)
 MDIO
 defined [56](#)
 MiCA
 defined [56](#)
 MKI field
 defined [52](#)
 MMIO
 defined [56](#)
 registers
 defined [56](#)
 mPIPE
 defined [52](#), [56](#)
 mPIPE driver
 initializing [16](#)
 MPL
 defined [52](#)
 MPS
 defined [52](#)
 MRS
 defined [52](#)
 MSB [52](#)
 MSB/MSBs

 defined [52](#)
 MSb/MSbs
 defined [52](#)
 MSW [52](#)
 MTU
 defined [52](#)

N

NAT
 defined [52](#)
 NAT-T
 defined [52](#)
 NH
 defined [52](#)
 notation conventions [ix](#)
 NotifGroup [3](#), [12](#), [13](#), [14](#), [18](#), [19](#), [21](#), [22](#)
 notification group [3](#)
 notification ring [3](#)
 NotifRing [3](#), [12](#), [13](#), [14](#), [18](#), [19](#), [20](#), [21](#), [22](#)
 defined [56](#)
 NPD
 defined [52](#)
 NPH
 defined [52](#)
 NVRAM
 defined [52](#)

O

ODT
 defined [52](#)
 OFB
 defined [52](#)
 operation
 defined [56](#)
 organization of this guide [vii](#)
 OTP
 defined [52](#)
 outbound data
 defined [56](#)

P

packets
 distributing [3](#)
 PCAP (packet capture) files [21](#)
 PCS
 defined [52](#)
 PD [52](#)
 defined [52](#)
 PH
 defined [52](#)
 PIO
 defined [52](#)
 interface
 defined [56](#)
 PIPE

- defined 56
- PKA
 - defined 52
- PKCP
 - defined 53
- PKE
 - defined 53
- PKI
 - defined 53
- Plaintext
 - defined 56
- PLD 56
 - defined 56
- priority queue
 - defined 57
- PRNG
 - defined 53
- Programmable Logic Device
 - See PLD
- promiscuous mode
 - defined 57
- Pseudo Random Number Generator
 - defined 56
- pull-down
 - See PD

Q

- QDN
 - defined 57
 - mesh interface 57

R

- RAM
 - defined 53
- RDN
 - defined 53
- red data
 - defined 57
- result
 - token
 - defined 57
- RGMI
 - defined 53
- ROM
 - defined 53
- RSA
 - defined 53
- RTP
 - defined 53

S

- SA 57
 - defined 53
- SDN
 - defined 53

- Serial RapidIO
 - See SRIO
- SHA
 - defined 53
- sim_enable_mpipe_links() 17, 19
- SOC
 - defined 53
- SPI
 - defined 53
- SPI-SROM
 - defined 53
- SPRAM
 - defined 53
- SQ
 - defined 53, 57
- SRAM
 - defined 53
- SRC
 - defined 53
- SRIO 57
 - defined 57
- SRTIP
 - defined 53
- standard-mode
 - defined 55, 57
- STI
 - defined 53
- support
 - technical or customer viii
- SWDS port
 - defined 53

T

- tag
 - defined 57
- TC
 - defined 53
- TCM
 - defined 53
- TCP
 - defined 53
- technical support viii
- text
 - defined 57
- TFC
 - defined 53
- tile-monitor 21
- TL
 - defined 53
- TLB
 - defined 53
- TLP
 - defined 53
- TLS
 - defined 53

- tmc_alloc_get_huge_pagesize() [12, 13, 14, 18, 21, 22, 23](#)
 - tmc_alloc_map() [12, 13, 14, 18, 21, 22, 23](#)
 - tmc_alloc_set_home() [12, 13, 14, 18, 20, 22, 23](#)
 - tmc_alloc_set_huge() [12, 13, 14, 18, 21, 22, 23](#)
 - tmc_alloc_set_pagesize() [12, 13, 14, 18, 20, 22, 23](#)
 - tmc_alloc_set_shared() [18](#)
 - tmc_cpus_count() [11, 12, 20, 22](#)
 - tmc_cpus_find_nth_cpu() [12, 13, 18, 22, 23](#)
 - tmc_cpus_get_my_affinity() [11, 12, 14, 18, 19, 20, 22](#)
 - tmc_sync_barrier_init() [18, 20](#)
 - tmc_task_watch_forked_children() [18](#)
 - tms_alloc_map() [19](#)
 - token
 - defined [57](#)
 - TOS
 - defined [53](#)
 - TRIO
 - defined [57](#)
 - TRIO_MAC_CONFIG register [52](#)
 - TRNG
 - defined [53](#)
 - TTL
 - defined [54](#)
- U**
- UART
 - defined [54, 57](#)
 - UDN
 - defined [54](#)
 - UDP
 - defined [54](#)
 - UL
 - defined [54](#)
- V**
- VC
 - defined [54](#)
 - VLIW architecture
 - defined [57](#)
 - VPP
 - defined [54](#)
- W**
- WEP
 - defined [54](#)

